COVID-19 Patient Health Prediction & Analytics

CLR204 Classification and Regression Analysis

Assessment 2

TASK 1**

By: `Jonathan Moses Nali (A00152708)`

There was a typo in the data.csv file in the symptom1 coloumn that i fixed manually from the start it was in line 875 and it was feve\ which i changed to fever (the dataset with the change was submitted along with the rest of the files, please use it as the main Data.csv file)

In [132…
```
# These are the libraries that we will be using
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# This is the data
dataset = pd.read_csv('data.csv')
dataset.duplicated().sum()

# This is a refererence that was used throughout the notebook
# Kelania, A. (2026, July 3). Data Science Tutorial. Geeksforgeeks. https://www.geeksforgeeks.org/data-science/data-science-for-beginners/
```

Out[132…
```
np.int64(0)
```

1. Dataset Description:

In [133…
```
import pandas as pd
import numpy as np


# This loads the CSV dataset file
dataset = pd.read_csv('data.csv')
# This displays the statistical summary of the datasaet
# Kattamuri, M. (2026, April 30). Exploratory Data Analysis in Python. Geeksforgeeks.https://www.geeksforgeeks.org/data-analysis/exploratory-data-analysis-in-python/
# Bhutani, K. (2025, July 26). Pandas DataFrame describe() Method. Geeksforgeeks. https://www.geeksforgeeks.org/pandas/python-pandas-dataframe-describe-method/
# Pydata. 2026. Essential basic functionality. Pydata. https://pandas.pydata.org/docs/user_guide/basics.html
print("\n" + "="*82)
print("SUMMARY OF THE DATASET")
print("="*82)
print(dataset.describe().to_string())



# This is a general overview of the dataset
# Pydata. 2026. pandas.DataFrame.shape. Pydata. https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.shape.html
# AAA. (2025, July 26). Python | pandas.DataFrame.info(). GeeksforGeeks. https://www.geeksforgeeks.org/python/python-pandas-dataframe-info/
# Jablonski, J. (2024, November 30). Python's F-String for String Interpolation and Formatting. Real Python. https://realpython.com/python-f-strings/
print("\n" + "="*45)
print("OVERVIEW OF THE DATASET")
print("="*45)
print(f"Total Patients Records (Rows): {dataset.shape[0]}")
print(f"Total Variables (Columns): {dataset.shape[1]}")
print("-"*45)
print("Data Types (Columns) & Non-empty Count")
print("-"*45)
dataset.info()

# Analysis of the Target Variable ("Death")
# AAA. (2025, July 11). Python | pandas.Series.value_counts(). GeeksforGeeks. https://www.geeksforgeeks.org/pandas/python-pandas-series-value_counts/
# Jablonski, J. (2024, November 30). Python's F-String for String Interpolation and Formatting. Real Python. https://realpython.com/python-f-strings/
death_counts = dataset['death'].value_counts()
```

```
death_pct = dataset['death'].value_counts(normalize=True) * 45
print("\n" + "="*45)
print("TARGET CLASS DISTRIBUTION ('death')")
print("="*45)
print(f"Recovered / Survived (Class 0): {death_counts[0]} ({death_pct[0]:.2f}%)")
print(f"Deceased (Class 1): {death_counts[1]} ({death_pct[1]:.2f}%)")

# Analysis of the missing values
# Kelania, A. (2025, November 11). Working with Missing Data in Pandas. Geeksforgeeks. https://www.geeksforgeeks.org/data-analysis/working-with-missing-data-in-pandas/
print("\n" + "="*45)
print("MISSING VALUES")
print("="*45)
missing_count = dataset.isnull().sum()
missing_pct = (dataset.isnull().sum() / len(dataset)) * 100
missing_df = pd.DataFrame({'Missing Count': missing_count,'Percentage (%)': missing_pct.round(2)})
print(missing_df[missing_df['Missing Count'] > 0].sort_values(by='Missing Count', ascending=False))

# This shows the first 7 Patient Records
dataset.head(7)
```

```
================================================================================
SUMMARY OF THE DATASET
================================================================================
                id         age    vis_wuhan   from_wuhan        death         recov
count  1085.000000  843.000000  1085.000000  1081.000000  1085.000000  1085.000000
mean    543.000000   49.483689     0.176959     0.144311     0.058065     0.146544
std     313.356825   18.255334     0.381810     0.351567     0.233973     0.353814
min       1.000000    0.250000     0.000000     0.000000     0.000000     0.000000
25%     272.000000   35.000000     0.000000     0.000000     0.000000     0.000000
50%     543.000000   51.000000     0.000000     0.000000     0.000000     0.000000
75%     814.000000   64.000000     0.000000     0.000000     0.000000     0.000000
max    1085.000000   96.000000     1.000000     1.000000     1.000000     1.000000

==========================================
OVERVIEW OF THE DATASET
==========================================
Total Patients Records (Rows): 1085
Total Variables (Columns): 17
------------------------------------------
Data Types (Columns) & Non-empty Count
------------------------------------------
<class 'pandas.DataFrame'>
RangeIndex: 1085 entries, 0 to 1084
Data columns (total 17 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   id          1085 non-null   int64
 1   location    1085 non-null   str
 2   country     1085 non-null   str
 3   gender      902 non-null    str
 4   age         843 non-null    float64
 5   sym_on      449 non-null    str
 6   hosp_vis    449 non-null    str
 7   vis_wuhan   1085 non-null   int64
 8   from_wuhan  1081 non-null   float64
 9   death       1085 non-null   int64
 10  recov       1085 non-null   int64
 11  symptom1    270 non-null    str
 12  symptom2    161 non-null    str
 13  symptom3    73 non-null     str
 14  symptom4    19 non-null     str
 15  symptom5    3 non-null      str
 16  symptom6    1 non-null      str
dtypes: float64(2), int64(4), str(11)
memory usage: 144.2 KB

==========================================
TARGET CLASS DISTRIBUTION ('death')
==========================================
Recovered / Survived (Class 0): 1022 (42.39%)
Deceased (Class 1): 63 (2.61%)

==========================================
MISSING VALUES
==========================================
            Missing Count  Percentage (%)
symptom6             1084           99.91
symptom5             1082           99.72
symptom4             1066           98.25
symptom3             1012           93.27
symptom2              924           85.16
symptom1              815           75.12
sym_on                636           58.62
hosp_vis              636           58.62
age                   242           22.30
gender                183           16.87
from_wuhan              4            0.37
```

Out[133…

	id	location	country	gender	age	sym_on	hosp_vis	vis_wuhan	from_wuhan	death	recov	symptom1	symptom2	symptom3	symptom4	symptom5	symptom6
0	1	Shenzhen, Guangdong	China	male	66.0	1/3/2020	1/11/2020	1	0.0	0	0	NaN	NaN	NaN	NaN	NaN	NaN
1	2	Shanghai	China	female	56.0	1/15/2020	1/15/2020	0	1.0	0	0	NaN	NaN	NaN	NaN	NaN	NaN
2	3	Zhejiang	China	male	46.0	1/4/2020	1/17/2020	0	1.0	0	0	NaN	NaN	NaN	NaN	NaN	NaN
3	4	Tianjin	China	female	60.0	NaN	NaN	1	0.0	0	0	NaN	NaN	NaN	NaN	NaN	NaN
4	5	Tianjin	China	male	58.0	NaN	NaN	0	0.0	0	0	NaN	NaN	NaN	NaN	NaN	NaN
5	6	Chongqing	China	female	44.0	NaN	NaN	0	1.0	0	0	NaN	NaN	NaN	NaN	NaN	NaN
6	7	Sichuan	China	male	34.0	NaN	NaN	0	1.0	0	0	NaN	NaN	NaN	NaN	NaN	NaN

Dataset description: GO THROUGH THIS AND FIX AND INCLUDE SOME OTHER IMPORTANT THINGS

- **Dataset size:** The "Covid-19-Patient-Health-Analytics" dataset has 1,085 patients records across 17 variables
- **Death and recovered variable:** There is a large difference in size of both variables, **recovered patients** have a value of **94.19%** which is approximately **1,022 patients**, the death toll however has a percentage of **5.81%** which is **63 individuals**
- **Target variable: Death** is the target variable. Death = 1 and Recovered = 0 in the dataset
- **Missing data:** These are the percentage and amount of missing variables in the dataset, `age` (22.30% or 242), `gender` (16.87% or 183), `hosp_vis` (58.62% or 636), `sym_on` (58.62% or 636), `from_wuhan` (0.37% or 4), `symptom 1` (75.12% or 815), `symptom 2` (85.15% or 924), `symptom 3` (93.27% or 1012), `symptom 4` (98.25% or 1066), `symptom 5` (99.72% or 1082), `symptom 6` (99.91% or 1084)
- This dataset was generated by SRK and obtained from kaggle.com, Atharva-Peshkar along with the authors fo the research paper have produced another dataset with the records of 222 patients which is used for the training model
- **Issues identified:** the error identified in the raw dataset was in the `symptom 1` column row 875 which is `id` 874, the error was the typo '`feve\`' instead of '`fever`', the issue identified was corrected before the data analysis

2. Data Analysis:

2.1 This is a list of the types of symptoms

In [134…

```
import pandas as pd

# These are the references for the data cleaning and analysis of the symptoms
# Bhuvaneswar005. (2022). Covid-19-detection-using-Machine-Learning. Github. https://github.com/Bhuvaneswar005/Covid-19-detection-using-Machine-Learning/blob/main/Covid-19%20detection%20using%20ML.ipynb
# NumPy. (2026). numpy.ravel. Numpy. https://numpy.org/doc/stable/reference/generated/numpy.ravel.html
# Bhutani, K. (2025, July 11). Pandas: Python Pandas Series str.strip(), lstrip() and rstrip(). GeeksforGeeks. https://www.geeksforgeeks.org/pandas/python-pandas-series-str-strip-lstrip-and-rstrip/
# Bhutani, K. (2023, August 23). Pandas: Python Pandas Series str.lower(), upper() and title(). GeeksforGeeks. https://www.geeksforgeeks.org/pandas/python-pandas-series-str-lower-upper-and-title/
# Pandas. (2026). pandas.DataFrame.to_string. Pandas Documentation. https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_string.html
# Pandas. (2026). pandas.Series.nunique. Pandas Documentation. https://pandas.pydata.org/docs/reference/api/pandas.Series.nunique.html
# Peshkar, A. (2020). Covid-19-Patient-Health-Analytics. Github. https://github.com/Peshkar/Covid-19-Patient-Health-Analytics

# This loads the dataset
dataset = pd.read_csv('data.csv')

# This extracts and cleans all the symptom columns (1 - 6)
sympt_cols = [f'symptom{i}' for i in range(1, 7)]
raw_sympt = dataset[sympt_cols].values.ravel()

cleaned_sympt = pd.Series(raw_sympt).dropna().astype(str).str.strip().str.lower()
cleaned_sympt = cleaned_sympt[cleaned_sympt != '']

# This calculates frequencies and converts to Data frame
raw_frequency_df = cleaned_sympt.value_counts().reset_index()
raw_frequency_df.columns = ['Symptom Name', 'Frequency']

# This formats the symptom names to Title Case for reporting
raw_frequency_df['Symptom Name'] = raw_frequency_df['Symptom Name'].str.title()

#This is to calculate the unique symptoms in the dataset and display the individual symptom frequencies
total_unique_symptoms = len(raw_frequency_df)

print(f"Total Unique Symptoms: {total_unique_symptoms}")

print("="*45)
print("  INDIVIDUAL SYMPTOM FREQUENCIES (RAW DATA)")
```

```
print("="*45)
print(raw_frequency_df.to_string(index=False))
```

```
Total Unique Symptoms: 50
=============================================
  INDIVIDUAL SYMPTOM FREQUENCIES (RAW DATA)
=============================================
          Symptom Name  Frequency
                 Fever        203
                 Cough        114
               Malaise         28
           Sore Throat         28
              Headache         15
                 Chills         13
            Runny Nose         11
               Fatigue         11
              Diarrhea         10
                Sputum         10
           Joint Pain          9
   Shortness Of Breath          9
               Vomiting         6
          Muscle Pain          4
               Dyspnea          4
     Throat Discomfort          4
             Pneumonia          3
  Difficulty Breathing          3
Difficult In Breathing          2
           Throat Pain          2
                  Cold          2
      Loss Of Appetite          2
        Breathlessness          2
                Nausea          2
            Chest Pain          2
            High Fever          2
          Muscle Aches          2
               Myalgia          2
        Abdominal Pain          1
       Nasal Discharge          1
  Respiratory Distress          1
            Mild Cough          1
                Sneeze          1
            Heavy Head          1
                Thirst          1
                   Flu          1
            Mild Fever          1
         Muscle Cramps          1
        Aching Muscles          1
             Sore Body          1
      Chest Discomfort          1
                 Chill          1
          Flu Symptoms          1
                Reflux          1
   Physical Discomfort          1
         Itchy Throat          1
             Coughing          1
                Tired          1
              Myalgias          1
     Cough With Sputum          1
```

2.2 COVID-19 Symptoms in patients **(Univariate)**

In [115…

```
import pandas as pd
import matplotlib.pyplot as plt

# Peshkar, A. (2020). Covid-19-Patient-Health-Analytics. Github. https://github.com/Peshkar/Covid-19-Patient-Health-Analytics
# This laods the dataset
dataset = pd.read_csv('data.csv')

# this converts all symptom columns into a single flat array
# https://numpy.org/doc/stable/reference/generated/numpy.ravel.html
symptom_cols = [c for c in dataset.columns if 'symptom' in c.lower()]
raw_symptoms = dataset[symptom_cols].values.ravel()

# This is the formatting (strip whitespace, convert to lowercase, drop NaNs)
cleaned_symptoms = pd.Series(raw_symptoms).dropna().astype(str).str.strip().str.lower()
```

```
cleaned_symptoms = cleaned_symptoms[cleaned_symptoms != '']

# This uses mapping to categorize the different symptoms into the 5 categories
# Some symptoms could not be classified so manual mapping was implemented to maek sure all the symptoms were categorized into their respective categories
# https://www.geeksforgeeks.org/python/python-pandas-map/
# https://www.w3resource.com/pandas/series/series-groupby.php
figure1_mapping = {

    # Category: Fever
    'fever': 'fever',
    'high fever': 'fever',
    'mild fever': 'fever',
    'feverish': 'fever',
    'pyrexia': 'fever',
    'chill': 'fever',
    'flu symptoms': 'fever',

    # Category: Cough
    'cough': 'cough',
    'dry cough': 'cough',
    'coughing': 'cough',
    'productive cough': 'cough',
    'sputum': 'cough',
    'pneumonia': 'cough',
    'pneumonitis': 'cough',
    'mild cough': 'cough',
    'cough with sputum': 'cough',
    'respiratory distress': 'cough',

    # Category: Cold
    'cold': 'cold',
    'flu': 'cold',
    'sore throat': 'cold',
    'throat pain': 'cold',
    'runny nose': 'cold',
    'nasal congestion': 'cold',
    'rhinorrhea': 'cold',
    'chills': 'cold',
    'sneeze': 'cold',
    'nasal discharge': 'cold',
    'throat discomfort': 'cold',
    'itchy throat': 'cold',
    'chest pain': 'cold',
    'chest discomfort': 'cold',
    'abdominal pain': 'cold',
    'thirst': 'cold',
    'reflux': 'cold',

    # Category: Fatigue and Body Pain
    'fatigue': 'Fatigue and Body Pain',
    'myalgia': 'Fatigue and Body Pain',
    'muscle pain': 'Fatigue and Body Pain',
    'body pain': 'Fatigue and Body Pain',
    'body aches': 'Fatigue and Body Pain',
    'joint pain': 'Fatigue and Body Pain',
    'headache': 'Fatigue and Body Pain',
    'breathlessness': 'Fatigue and Body Pain',
    'difficult in breathing': 'Fatigue and Body Pain',
    'difficulty breathing': 'Fatigue and Body Pain',
    'shortness of breath': 'Fatigue and Body Pain',
    'dyspnea': 'Fatigue and Body Pain',
    'aching muscles': 'Fatigue and Body Pain',
    'muscle aches': 'Fatigue and Body Pain',
    'sore body': 'Fatigue and Body Pain',
    'myalgias': 'Fatigue and Body Pain',
    'muscle cramps': 'Fatigue and Body Pain',

    # Category: Malaise
    'malaise': 'malaise',
    'loss of appetite': 'malaise',
    'anorexia': 'malaise',
    'diarrhea': 'malaise',
    'nausea': 'malaise',
    'vomiting': 'malaise',
    'tired': 'malaise',
```

```
    'physical discomfort': 'malaise',
    'heavy head': 'malaise',
}

# This assisgns the different symptoms to their respective categories based on the mapping above
mapped_symptoms = cleaned_symptoms.map(figure1_mapping)

# This is a check for unmapped symptoms
# https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.isna.html#pandas.DataFrame.isna
# https://www.geeksforgeeks.org/python/python-pandas-dataframe-isna/
# https://www.w3resource.com/pandas/series/series-unique.php
unmapped = cleaned_symptoms[mapped_symptoms.isna()]
if len(unmapped) > 0:
    print(f"Warning: Found {len(unmapped)} unmapped symptom entries in data.csv:")
    print(unmapped.unique())
else:
    print ('All symptoms successfylly accounted for in the 5 categories')

# This counts the frequency of the mapped symptoms and creates a summary (frequency) that is displayed
# https://pandas.pydata.org/docs/reference/api/pandas.Series.value_counts.html
figure1_order = ['malaise', 'Fatigue and Body Pain', 'cold', 'cough', 'fever']
category_counts = mapped_symptoms.value_counts().reindex(figure1_order).fillna(0).astype(int)

summary_df = pd.DataFrame({
    'Figure 1 Category': category_counts.index,
    'Occurrences': category_counts.values
})

print("\n" + "=" * 50)
print("       MAIN RAW DATASET CATEGORY SUMMARY:")
print("="*50)
print(summary_df.to_string(index=False))



# This is for the bar chart that is displayed for the 5 categories of symptoms
# https://www.geeksforgeeks.org/python/countplot-using-seaborn-in-python/
# https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.subplots.html
# https://www.baeldung.com/cs/matplotlib-add-value-labels-bar-plot
# https://www.geeksforgeeks.org/python/adding-value-labels-on-a-matplotlib-bar-chart/
fig, ax = plt.subplots(figsize=(8, 6))

bars = ax.bar(summary_df['Figure 1 Category'], summary_df['Occurrences'], color='blue', width=0.8)
ax.plot(summary_df['Figure 1 Category'], summary_df['Occurrences'], color='red', marker='D', linewidth=1.5, markersize=6)

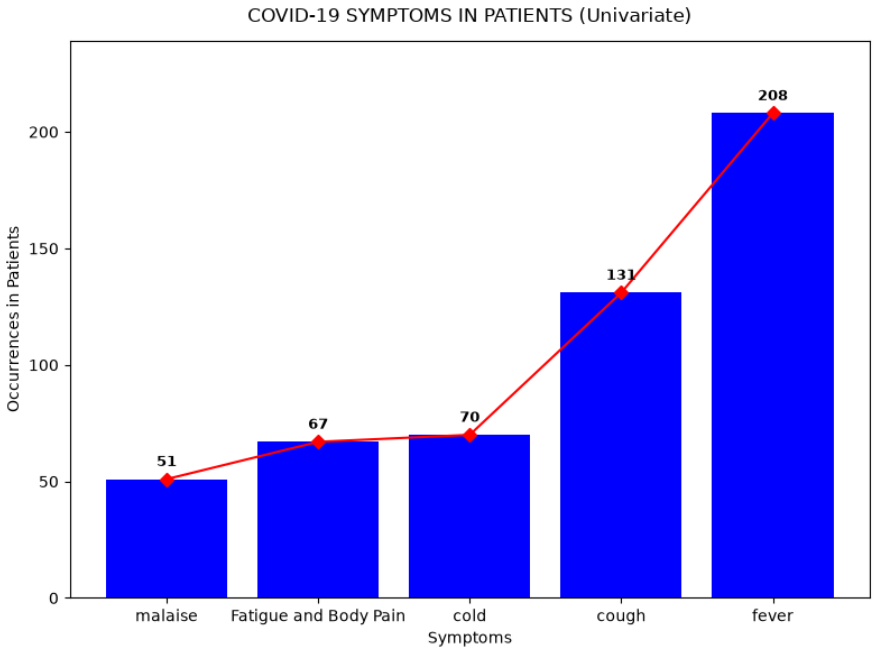
ax.set_title('COVID-19 SYMPTOMS IN PATIENTS (Univariate)', fontsize=12, pad=12)
ax.set_xlabel('Symptoms', fontsize=10)
ax.set_ylabel('Occurrences in Patients', fontsize=10)
ax.set_ylim(0, summary_df['Occurrences'].max() * 1.15)

# This adds the value labels on top of each bar in the bar chart
for bar in bars:
    yval = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2.0, yval + (summary_df['Occurrences'].max() * 0.02),
            int(yval), ha='center', va='bottom', fontsize=9, fontweight='bold')

plt.tight_layout()
plt.show()
```

```
All symptoms successfylly accounted for in the 5 categories

==================================================
       MAIN RAW DATASET CATEGORY SUMMARY:
==================================================
    Figure 1 Category  Occurrences
              malaise           51
Fatigue and Body Pain           67
                 cold           70
                cough          131
                fever          208
```



2.3 Patient outcome counts by gender & Patient outcomes by age group Plots **(Bivariate)**

In [136]:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# This loads the dataset
# These are the references that were used to create the plots:
# Rai. A, (2025, July 15). seaborn.countplot() in Python. Geeksforgeeks. https://www.geeksforgeeks.org/python/countplot-using-seaborn-in-python/
# Matplotlib, (2026). matplotlib.pyplot.subplots. https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.subplots.html
# Zvornicanin, E. (2025, April 29). How to Add Value Labels to a Bar Plot in Matplotlib?. Baeldung. https://www.baeldung.com/cs/matplotlib-add-value-labels-bar-plot
# Shivansh. (2025, July 23). Adding value labels on a Matplotlib Bar Chart. Geeksforgeeks.https://www.geeksforgeeks.org/python/adding-value-labels-on-a-matplotlib-bar-chart/
dataset = pd.read_csv('data.csv')

# This makes sure the coloumns are properly formatted for the analysis
dataset['age'] = pd.to_numeric(dataset['age'], errors='coerce')
dataset['death'] = pd.to_numeric(dataset['death'], errors='coerce').fillna(0).astype(int)

# This creates age brackets of 20 years
bins = [0, 20, 40, 60, 80, 110]
labels = ['0-19', '20-39', '40-59', '60-79', '80+']
dataset['Age Group'] = pd.cut(dataset['age'], bins=bins, labels=labels, right=False)

# This is for the outcome key in the plots
dataset['Outcome'] = dataset['death'].map({0: 'Survived (0)', 1: 'Deceased (1)'})
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

#This is for the first plot 'PATIENT OUTCOME COUNTS BY GENDER (Bivariate)'
df_gender = dataset.dropna(subset=['gender'])

sns.countplot(
    data=df_gender, x='gender', hue='Outcome', palette={'Survived (0)': 'red', 'Deceased (1)': 'blue'}, ax=axes[0], alpha=0.9
)
axes[0].set_title('PATIENT OUTCOME COUNTS BY GENDER (Bivariate)', fontsize=12, fontweight='bold', pad=12)
axes[0].set_xlabel('Gender', fontsize=11)
axes[0].set_ylabel('Patient Count', fontsize=11)
axes[0].legend(title='Outcome', loc='upper right')

# Expand y-axis top limit
```

```
max_gender_count = df_gender.groupby(['gender', 'Outcome'], observed=False).size().max()
axes[0].set_ylim(0, max_gender_count * 1.15)

# Add exact numeric count labels above each bar
for p in axes[0].patches:
    h = p.get_height()
    if not np.isnan(h) and h > 0:
        axes[0].annotate(
            f'{int(h)}', (p.get_x() + p.get_width() / 2., h), ha='center', va='bottom', fontsize=10, fontweight='bold', xytext=(0, 3), textcoords='offset points'
        )

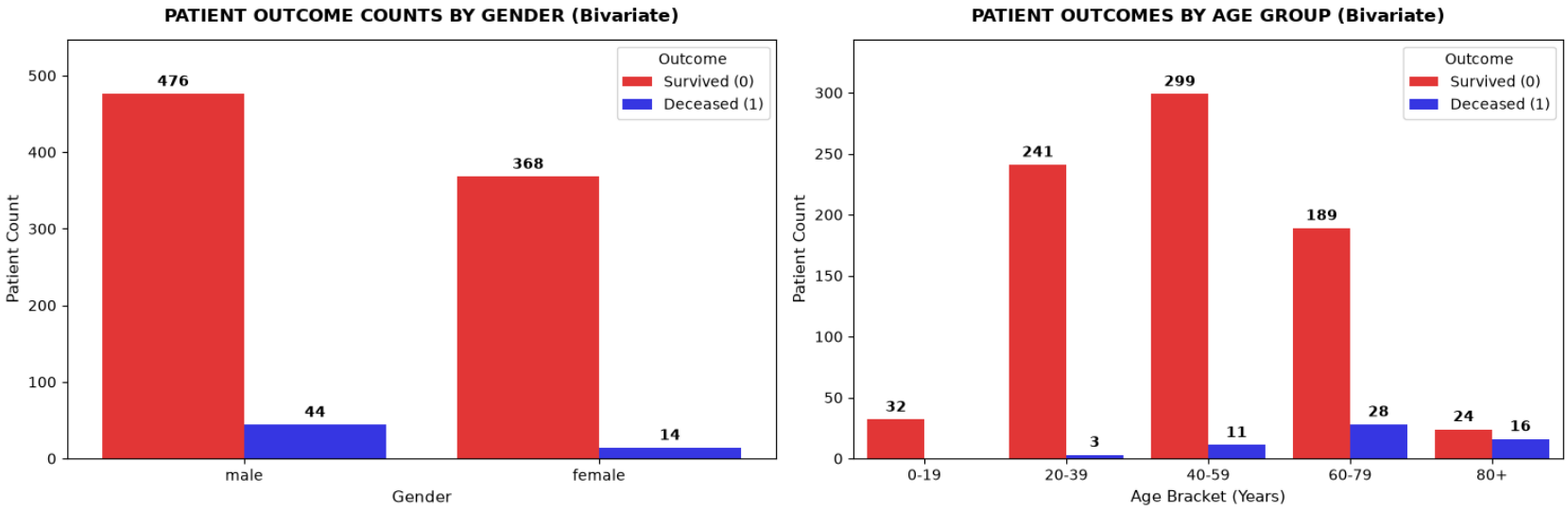
# This is for the second plot 'PATIENT OUTCOMES BY AGE GROUP (Bivariate)'
df_age = dataset.dropna(subset=['Age Group'])

sns.countplot(
    data=df_age, x='Age Group', hue='Outcome', palette={'Survived (0)': 'red', 'Deceased (1)': 'blue'}, ax=axes[1], alpha=0.9
)
axes[1].set_title('PATIENT OUTCOMES BY AGE GROUP (Bivariate)', fontsize=12, fontweight='bold', pad=12)
axes[1].set_xlabel('Age Bracket (Years)', fontsize=11)
axes[1].set_ylabel('Patient Count', fontsize=11)
axes[1].legend(title='Outcome', loc='upper right')

# This expands the y-axis top limit for the age group plot to ensure that the count labels fit within the plot area
max_age_count = df_age.groupby(['Age Group', 'Outcome'], observed=False).size().max()
axes[1].set_ylim(0, max_age_count * 1.15)

# This adds the value labels on top of each bar in the age group plot
for p in axes[1].patches:
    h = p.get_height()
    if not np.isnan(h) and h > 0:
        axes[1].annotate(
            f'{int(h)}', (p.get_x() + p.get_width() / 2., h), ha='center', va='bottom', fontsize=10, fontweight='bold', xytext=(0, 3), textcoords='offset points'
        )

plt.tight_layout()
plt.show()
```



Dataset analysis:

- **Symptoms:** The analysis resulted in the identification of 50 individual symptoms, they are classified into 5 categories, `Malaise`, ` Fatigue and body pain`, `Cold symptoms`, `Cough` and `Fever`. There was an error in the symptoms 1 column, but that typo was correct initially. There were only 527 patients symptoms recorded due to patients only experiencing 1, 2 or 3 symptoms only (out of the 6).
- **Major Symptoms categories:** 208 patients had `Fever` symptoms, 131 patients had `Cough` symptoms, 70 patients had `Cold Symptoms`, 67 patients had `Fatigue and body pain` symptoms and 51 patients had `Malaise` symptoms.

- **Patient outcome counts by gender:** The `Gender` column in the raw dataset has 183 missing values which The plots visually exposes. The analysis identified that 476 male along with 368 female patients survived COVID-19 while 44 male and 14 female patients passed away due to the virus.
- **Patient outcomes by age group:** The `Age` column in the raw dataset has 242 missing values. Majority of patients who survived the virus are between the ages $20 - 79$ while the majority that did not survive it were in the age range of $40 - 89$

3. Data Pre-processing & Correlation:

3.1 Data Pre-processing

In [137...

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder

# These are the references
# Pykes, K. (2025, January 15). Data Preprocessing: A Complete Guide with Python Examples. Datacamp. https://www.datacamp.com/blog/data-preprocessing
# Jain, D. (2026, February 7). Data Preprocessing in Data Mining. Geeksforgeeks.  https://www.geeksforgeeks.org/data-science/data-preprocessing-in-data-mining/

# Data preprocessing and cleaning for the COVID-19 patient dataset
# This loads the raw dataset
raw_df = pd.read_csv('data.csv')
df = raw_df.copy()

print("="*50)
print("Original Data Shape:", df.shape)

# This is for the Date Parsing & the delay inhospital visits after symptom onset calculation
df['sym_on'] = pd.to_datetime(df['sym_on'], errors='coerce')
df['hosp_vis'] = pd.to_datetime(df['hosp_vis'], errors='coerce')

df['diff_sym_hos'] = (df['hosp_vis'] - df['sym_on']).dt.days
df['diff_sym_hos'] = df['diff_sym_hos'].apply(lambda x: x if x >= 0 else np.nan)
df['diff_sym_hos'] = df['diff_sym_hos'].fillna(df['diff_sym_hos'].median())

# This is for Numerical & Binary Processing for the age and binary columns
df['age'] = pd.to_numeric(df['age'], errors='coerce')
df['age'] = df['age'].fillna(df['age'].median())

binary_cols = ['vis_wuhan', 'from_wuhan', 'death', 'recov']
for col in binary_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0).astype(int)

# This is for the label encoding of the categorical columns and text normalization
cat_cols = [
    'location', 'country', 'gender',
    'symptom1', 'symptom2', 'symptom3',
    'symptom4', 'symptom5', 'symptom6'
]

le = LabelEncoder()
for col in cat_cols:
    if col in df.columns:
        df[col] = df[col].astype(str).str.strip().str.lower()
        df[col] = df[col].replace({'nan': 'na', '': 'na', 'none': 'na', 'null': 'na'}).fillna('na')
        df[col] = le.fit_transform(df[col])

#  THis is for the make the final deduplicated feature selection (the 15 Core Features)
features = [
    'location', 'country', 'gender', 'age',
    'vis_wuhan', 'from_wuhan', 'death', 'recov',
    'symptom1', 'symptom2', 'symptom3', 'symptom4',
    'symptom5', 'symptom6', 'diff_sym_hos'
]

cleaned_df = df[features].copy()

# Automatically save the preprocessed data file
cleaned_df.to_csv('cleaned_data.csv', index=False)
```

```
print("Preprocessed Data Shape:", cleaned_df.shape)
print(" ")
print("="*50)
print(" Missing Value Count After Preprocessing ")
print("="*50)
print(cleaned_df.isnull().sum())
print(" ")
print("="*50)
print("First 10 rows of Cleaned Dataset:")
print("="*50)
print(cleaned_df.head(10).to_string())
```

```
==================================================
Original Data Shape: (1085, 17)
Preprocessed Data Shape: (1085, 15)

==================================================
 Missing Value Count After Preprocessing
==================================================
location        0
country         0
gender          0
age             0
vis_wuhan       0
from_wuhan      0
death           0
recov           0
symptom1        0
symptom2        0
symptom3        0
symptom4        0
symptom5        0
symptom6        0
diff_sym_hos    0
dtype: int64

==================================================
First 10 rows of Cleaned Dataset:
==================================================
   location  country  gender   age  vis_wuhan  from_wuhan  death  recov  symptom1  symptom2  symptom3  symptom4  symptom5  symptom6  diff_sym_hos
0       116        8       1  66.0          1           0      0      0        16        21        16         8         1         1           8.0
1       113        8       0  56.0          0           1      0      0        16        21        16         8         1         1           0.0
2       154        8       1  46.0          0           1      0      0        16        21        16         8         1         1          13.0
3       129        8       0  60.0          1           0      0      0        16        21        16         8         1         1           2.0
4       129        8       1  58.0          0           0      0      0        16        21        16         8         1         1           2.0
5        24        8       0  44.0          0           1      0      0        16        21        16         8         1         1           2.0
6       117        8       1  34.0          0           1      0      0        16        21        16         8         1         1           2.0
7        13        8       1  37.0          1           0      0      0        16        21        16         8         1         1           6.0
8        13        8       1  39.0          1           0      0      0        16        21        16         8         1         1           5.0
9        13        8       1  56.0          1           0      0      0        16        21        16         8         1         1           4.0
```

3.2 Correlation

In [138...

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder

# Bothma, J. (2024, December 13). Seaborn Heatmaps: A Guide to Data Visualization. https://www.datacamp.com/tutorial/seaborn-heatmaps
# Dhingra. F, (2026, July 17). Create a Correlation Matrix using Python. Geeksforgeeks. https://www.geeksforgeeks.org/data-science/create-a-correlation-matrix-using-python/

# Correlation Analysis and Heatmap Visualization
# Compute Pearson Correlation Matrix directly from the cleaned dataframe
corr_matrix = cleaned_df.corr(method='pearson')
print(" ")
print("="*50)
print(" Correlation Matrix ")
print("="*50)
print(corr_matrix.round(5).to_string())

# Plot Heatmap matching the study's visual layout and pink/red palette
plt.figure(figsize=(14, 12))

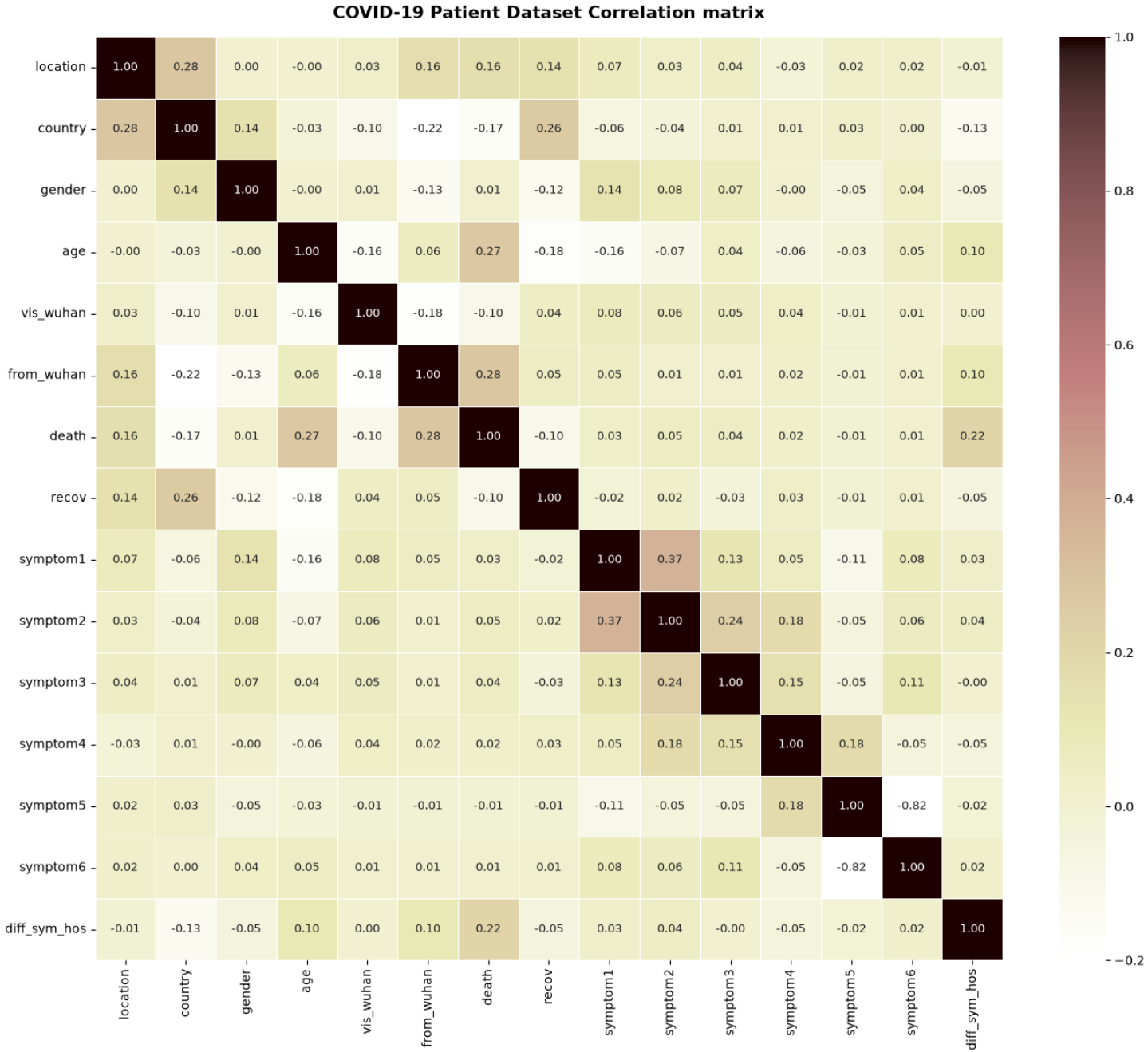
sns.heatmap( corr_matrix, annot=True,
```

```
    cmap='pink_r',
    vmin=-0.2,
    vmax=1.0,
    fmt=".2f",
    linewidths=0.5,
    cbar=True,
    annot_kws={"size": 9}
)

plt.title('COVID-19 Patient Dataset Correlation matrix', fontsize=14, fontweight='bold', pad=15)
plt.xticks(rotation=90, fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.tight_layout()

# This saves the heatmap as a high-resolution PNG file into the current folder and displays the heatmap
plt.savefig('cleaned_correlation_matrix.png', dpi=300, bbox_inches='tight')
plt.show()
```

```
==================================================
 Correlation Matrix
==================================================
              location  country   gender      age  vis_wuhan  from_wuhan     death     recov  symptom1  symptom2  symptom3  symptom4  symptom5  symptom6  diff_sym_hos
location       1.00000  0.28084  0.00018 -0.00024    0.02566     0.16173  0.16025  0.13977   0.06849   0.03075   0.04096  -0.02972   0.01839   0.01664      -0.00575
country        0.28084  1.00000  0.14012 -0.03234   -0.10245    -0.21687 -0.16550  0.26002  -0.05992  -0.03521   0.01304   0.00693   0.02524   0.00054      -0.13218
gender         0.00018  0.14012  1.00000 -0.00067    0.00766    -0.13319  0.01443 -0.11515   0.13705   0.08356   0.07496  -0.00094  -0.04678   0.03554      -0.05193
age           -0.00024 -0.03234 -0.00067  1.00000   -0.16428     0.05807  0.26794 -0.17601  -0.16018  -0.07474   0.03873  -0.05843  -0.02900   0.04684       0.09706
vis_wuhan      0.02566 -0.10245  0.00766 -0.16428    1.00000    -0.18313 -0.10480  0.04004   0.07539   0.05691   0.05483   0.03799  -0.01150   0.01408       0.00115
from_wuhan     0.16173 -0.21687 -0.13319  0.05807   -0.18313     1.00000  0.28015  0.04560   0.04642   0.01049   0.01256   0.01719  -0.01016   0.01245       0.09594
death          0.16025 -0.16550  0.01443  0.26794   -0.10480     0.28015  1.00000 -0.10288   0.03427   0.05019   0.04485   0.02034  -0.00616   0.00754       0.21611
recov          0.13977  0.26002 -0.11515 -0.17601    0.04004     0.04560 -0.10288  1.00000  -0.01762   0.02129  -0.03348   0.03395  -0.01027   0.01259      -0.05225
symptom1       0.06849 -0.05992  0.13705 -0.16018    0.07539     0.04642  0.03427 -0.01762   1.00000   0.37094   0.12529   0.05173  -0.10553   0.07665       0.02928
symptom2       0.03075 -0.03521  0.08356 -0.07474    0.05691     0.01049  0.05019  0.02129   0.37094   1.00000   0.24131   0.17697  -0.04516   0.05532       0.04445
symptom3       0.04096  0.01304  0.07496  0.03873    0.05483     0.01256  0.04485 -0.03348   0.12529   0.24131   1.00000   0.14793  -0.05407   0.10539      -0.00148
symptom4      -0.02972  0.00693 -0.00094 -0.05843    0.03799     0.01719  0.02034  0.03395   0.05173   0.17697   0.14793   1.00000   0.17606  -0.04986      -0.04873
symptom5       0.01839  0.02524 -0.04678 -0.02900   -0.01150    -0.01016 -0.00616 -0.01027  -0.10553  -0.04516  -0.05407   0.17606   1.00000  -0.81637      -0.01566
symptom6       0.01664  0.00054  0.03554  0.04684    0.01408     0.01245  0.00754  0.01259   0.07665   0.05532   0.10539  -0.04986  -0.81637   1.00000       0.01918
diff_sym_hos  -0.00575 -0.13218 -0.05193  0.09706    0.00115     0.09594  0.21611 -0.05225   0.02928   0.04445  -0.00148  -0.04873  -0.01566   0.01918       1.00000
```



Data Pre-processing & Correlation:

- **missing values and messsy data:** The dataset contained alot of missing values, majority of them were from the `symptoms`, `symp_on` and the `hosp_vis` variables (they were missing around 60 - 99 percent of the data)

- **Imbalance classes:** The target variable of the dataset was extremely skewed (it made up less than 5 percent)
- **Data conversion:** The variables like `gender` and `symptoms` had to be converted from text variables into numbers for the model to processes them
- **Correlations:** The correlation matrices were used to see how features related to each other which y proved to be mostly identical compared to the study. The important variables were identical but the other nonimportant variables displayed different values due to the preprocessing that affected thir values

4. Evaluation Metrics:

The same references were used accross the machine learning model training and evaluations

4.1 Logistic regression

In [139...

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import recall_score as rs, precision_score as ps, f1_score as fs
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix

# Same references were used throughout the evaluation metrics section
# These are the references for the boosted random forest and the evaluation metrics
# Aman, S. (2026, March 30). Evaluation Metrics in Machine Learning . Geeksforgeeks. https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/
# Peshkar, A. (2020). Covid-19-Patient-Health-Analytics. Github. https://github.com/Peshkar/Covid-19-Patient-Health-Analytics
# Scikit. (2026). 3.4. Metrics and scoring: quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html
# Scikit. (2026). DecisionTreeClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html
# Scikit. (2026). train_test_split. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html
# shahjamanbadhon.(2025). Machine-learning-practice. Github. https://github.com/shahjamanbadhon/machine-learning-practice/blob/main/Confusion_Matrix.ipynb
# W3Schools. (2026). Python Machine Learning - Confusion Matrix. https://www.w3schools.com/python/python_ml_confusion_matrix.asp
# Sharma, A. (2026, January 23). Understanding the Confusion Matrix in Machine Learning. Geeksforgeeks. https://www.geeksforgeeks.org/machine-learning/confusion-matrix-machine-learning/
# Galarnyk, M. (2025, August 4). Train Test Split: What It Means and How to Use It. Built In. https://builtin.com/data-science/train-test-split

tdata = pd.read_csv('train.csv')

# This is to calculate the diff_sym_hos from symptom onset and hospital visit dates
tdata['sym_on'] = pd.to_datetime(tdata['sym_on'], errors='coerce')
tdata['hosp_vis'] = pd.to_datetime(tdata['hosp_vis'], errors='coerce')
tdata['diff_sym_hos'] = (tdata['hosp_vis'] - tdata['sym_on']).dt.days
tdata['diff_sym_hos'] = tdata['diff_sym_hos'].apply(lambda x: x if x >= 0 else np.nan)
tdata['diff_sym_hos'] = tdata['diff_sym_hos'].fillna(tdata['diff_sym_hos'].median())

# This is for the numerical & Binary Processing
tdata['age'] = pd.to_numeric(tdata['age'], errors='coerce')
tdata['age'] = tdata['age'].fillna(tdata['age'].median())

binary_cols = ['vis_wuhan', 'from_wuhan', 'death', 'recov']
for col in binary_cols:
    if col in tdata.columns:
        tdata[col] = pd.to_numeric(tdata[col], errors='coerce').fillna(0).astype(int)

# This is for categorical text normalization & label encoding
cat_cols = ['location', 'country', 'gender', 'symptom1', 'symptom2', 'symptom3', 'symptom4', 'symptom5', 'symptom6']
le = LabelEncoder()
for col in cat_cols:
    if col in tdata.columns:
        tdata[col] = tdata[col].astype(str).str.strip().str.lower()
        tdata[col] = tdata[col].replace({'nan': 'na', '': 'na', 'none': 'na', 'null': 'na'}).fillna('na')
        tdata[col] = le.fit_transform(tdata[col])

# This is the model training and evaluation using the Logistic Regression Classifier
X = tdata[['location', 'country', 'gender', 'age', 'vis_wuhan', 'from_wuhan', 'symptom1', 'symptom2', 'symptom3', 'symptom4', 'symptom5', 'symptom6', 'diff_sym_hos']]
Y = tdata['death']

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=10)

# This initializes and fits the Logistic Regression Classifier
classifier = LogisticRegression(max_iter=1000, random_state=42)
classifier.fit(X_train, np.array(Y_train).reshape(Y_train.shape[0], 1).ravel())
```

```python
# This is for the Evaluation of the model
pred = np.array(classifier.predict(X_test))

recall_lr = rs(Y_test, pred, zero_division=0)
precision_lr = ps(Y_test, pred, zero_division=0)
f1_lr = fs(Y_test, pred, zero_division=0)
ma_lr = classifier.score(X_test, Y_test)

print(" ")
print("="*70)
print(' Evaluation metrics for test dataset (Logistic Regression) ')
print("="*70)
print('Recall Score: ', recall_lr)
print('Precision Score: ', precision_lr)
print('F1 Score: ', f1_lr)
print('Accuracy: ', ma_lr)

a = pd.DataFrame(Y_test)
a['pred'] = classifier.predict(X_test)
print('\n\tTable 3\n')
print(a.head())

# This Calculates and extracts the Confusion Matrix (TP, TN, FP, FN)
cm_lr = confusion_matrix(Y_test, pred)
TN_lr, FP_lr, FN_lr, TP_lr = cm_lr.ravel()

print(" ")
print("="*50)
print(' Confusion Matrix (Logistic Regression) ')
print("="*50)
print(f"True Negatives (TN):          {TN_lr}")
print(f"False Positives (FP):         {FP_lr}")
print(f"False Negatives (FN):         {FN_lr}")
print(f"True Positives (TP):          {TP_lr}")

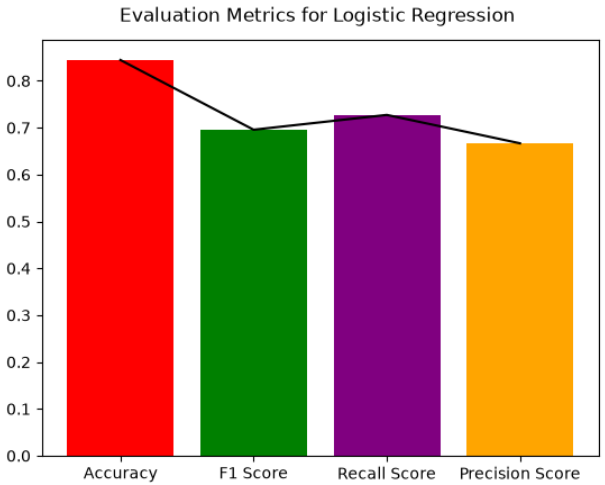
# This is for the plot
plt.bar(['Accuracy','F1 Score','Recall Score','Precision Score'],[ma_lr,f1_lr,recall_lr,precision_lr],color=['red','green','purple','orange'])
plt.plot([ma_lr,f1_lr,recall_lr,precision_lr],color='black')
plt.title('Evaluation Metrics for Logistic Regression ',fontsize=12, pad=12)
plt.show()
```

```
======================================================================
 Evaluation metrics for test dataset (Logistic Regression)
======================================================================
Recall Score:  0.7272727272727273
Precision Score:  0.6666666666666666
F1 Score:  0.6956521739130435
Accuracy:  0.8444444444444444

        Table 3

     death  pred
184      0     0
170      1     0
142      0     0
182      0     0
49       1     1

==================================================
 Confusion Matrix (Logistic Regression)
==================================================
True Negatives (TN):          30
False Positives (FP):         4
False Negatives (FN):         3
True Positives (TP):          8
```



4.2 Descision tree

In [148…

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier as dtc
from sklearn.metrics import recall_score as rs, precision_score as ps, f1_score as fs
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix

# These are the references for the Decision Tree Classifier and the evaluation metrics
# Aman, S. (2026, March 30). Evaluation Metrics in Machine Learning . Geeksforgeeks. https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/
# Peshkar, A. (2020). Covid-19-Patient-Health-Analytics. Github. https://github.com/Peshkar/Covid-19-Patient-Health-Analytics
# Scikit. (2026). 3.4. Metrics and scoring: quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html
# Scikit. (2026). DecisionTreeClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html
# Scikit. (2026). train_test_split. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html
# shahjamanbadhon.(2025). Machine-learning-practice. Github. https://github.com/shahjamanbadhon/machine-learning-practice/blob/main/Confusion_Matrix.ipynb
# W3Schools. (2026). Python Machine Learning - Confusion Matrix. https://www.w3schools.com/python/python_ml_confusion_matrix.asp
# Sharma, A. (2026, January 23). Understanding the Confusion Matrix in Machine Learning. Geeksforgeeks. https://www.geeksforgeeks.org/machine-learning/confusion-matrix-machine-learning/
# Galarnyk, M. (2025, August 4). Train Test Split: What It Means and How to Use It. Built In. https://builtin.com/data-science/train-test-split

# Load your master dataset (change 'data.csv' to train.csv)
tdata = pd.read_csv('train.csv')

# This is to calculate the diff_sym_hos from symptom onset and hospital visit dates
tdata['sym_on'] = pd.to_datetime(tdata['sym_on'], errors='coerce')
tdata['hosp_vis'] = pd.to_datetime(tdata['hosp_vis'], errors='coerce')
tdata['diff_sym_hos'] = (tdata['hosp_vis'] - tdata['sym_on']).dt.days
tdata['diff_sym_hos'] = tdata['diff_sym_hos'].apply(lambda x: x if x >= 0 else np.nan)
tdata['diff_sym_hos'] = tdata['diff_sym_hos'].fillna(tdata['diff_sym_hos'].median())

# This is for the numerical & Binary Processing
tdata['age'] = pd.to_numeric(tdata['age'], errors='coerce')
tdata['age'] = tdata['age'].fillna(tdata['age'].median())

binary_cols = ['vis_wuhan', 'from_wuhan', 'death', 'recov']
for col in binary_cols:
    if col in tdata.columns:
        tdata[col] = pd.to_numeric(tdata[col], errors='coerce').fillna(0).astype(int)

# This is for the label encoding and text normalization of the categorical columns
cat_cols = ['location', 'country', 'gender', 'symptom1', 'symptom2', 'symptom3', 'symptom4', 'symptom5', 'symptom6']
le = LabelEncoder()
for col in cat_cols:
    if col in tdata.columns:
        tdata[col] = tdata[col].astype(str).str.strip().str.lower()
        tdata[col] = tdata[col].replace({'nan': 'na', '': 'na', 'none': 'na', 'null': 'na'}).fillna('na')
```

```
        tdata[col] = le.fit_transform(tdata[col])

# Th is is the Decision Tree Classifier that is used to predict the death outcome based on the 15 core features
X = tdata[['location', 'country', 'gender', 'age', 'vis_wuhan', 'from_wuhan', 'symptom1', 'symptom2', 'symptom3', 'symptom4', 'symptom5', 'symptom6', 'diff_sym_hos']]
Y = tdata['death']

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=10)

# This initializes and fits the Decision Tree Classifier
classifier = dtc(max_depth=2)
classifier.fit(X_train, np.array(Y_train).reshape(Y_train.shape[0], 1))

# This is the evaluation of the model
pred = np.array(classifier.predict(X_test))

recall_dtc = rs(Y_test, pred, zero_division=0)
precision_dtc = ps(Y_test, pred, zero_division=0)
f1_dtc = fs(Y_test, pred, zero_division=0)
ma_dtc = classifier.score(X_test, Y_test)

print(" ")
print("="*60)
print('Evaluation metrics for test dataset (descision tree)')
print("="*60)
print('Recall Score: ', recall_dtc)
print('Precision Score: ', precision_dtc)
print('F1 Score: ', f1_dtc)
print('Accuracy: ', ma_dtc)

a = pd.DataFrame(Y_test)
a['pred'] = classifier.predict(X_test)
print('\n\tTable 3\n')
print(a.head())

# This calculates adn extracts the Confusion Matrix values
cm_dt = confusion_matrix(Y_test, pred)
TN_dt, FP_dt, FN_dt, TP_dt = cm_dt.ravel()

print(" ")
print("="*50)
print(' Confusion Matrix (Decision Tree) ')
print("="*50)
print(f"True Negatives (TN):          {TN_dt}")
print(f"False Positives (FP):         {FP_dt}")
print(f"False Negatives (FN):         {FN_dt}")
print(f"True Positives (TP):          {TP_dt}")

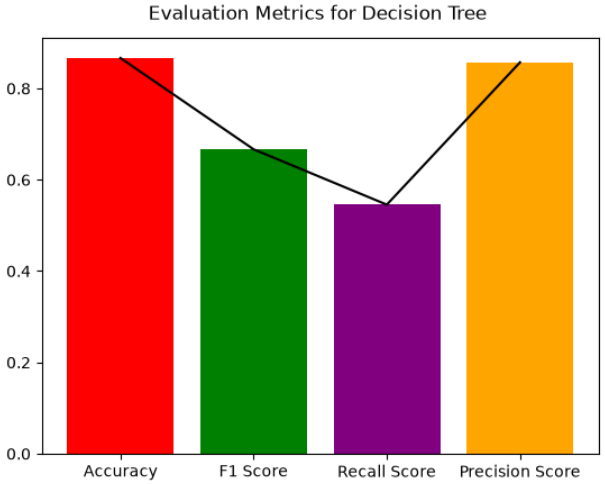
# This is for the plot
plt.bar(['Accuracy','F1 Score','Recall Score','Precision Score'],[ma_dtc,f1_dtc,recall_dtc,precision_dtc],color=['red','green','purple','orange'])
plt.plot([ma_dtc,f1_dtc,recall_dtc,precision_dtc],color='black')
plt.title('Evaluation Metrics for Decision Tree ',fontsize=12, pad=12)
plt.show()
```

```
============================================================
Evaluation metrics for test dataset (descision tree)
============================================================
Recall Score:  0.5454545454545454
Precision Score:  0.8571428571428571
F1 Score:  0.6666666666666666
Accuracy:  0.8666666666666667

        Table 3

     death  pred
184      0     0
170      1     0
142      0     0
182      0     0
49       1     0

================================================
 Confusion Matrix (Decision Tree)
================================================
True Negatives (TN):          33
False Positives (FP):         1
False Negatives (FN):         5
True Positives (TP):          6
```



4.3 Boosted Random Forest

In [141…

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier, RandomForestClassifier
from sklearn.metrics import recall_score as rs, precision_score as ps, f1_score as fs
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix

# Same references were used throughout the evaluation metrics section
# These are the references for the boosted random forest and the evaluation metrics
# Aman, S. (2026, March 30). Evaluation Metrics in Machine Learning . Geeksforgeeks. https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/
# Peshkar, A. (2020). Covid-19-Patient-Health-Analytics. Github. https://github.com/Peshkar/Covid-19-Patient-Health-Analytics
# Scikit. (2026). 3.4. Metrics and scoring: quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html
# Scikit. (2026). DecisionTreeClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html
# Scikit. (2026). train_test_split. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html
# shahjamanbadhon.(2025). Machine-learning-practice. Github. https://github.com/shahjamanbadhon/machine-learning-practice/blob/main/Confusion_Matrix.ipynb
# W3Schools. (2026). Python Machine Learning - Confusion Matrix. https://www.w3schools.com/python/python_ml_confusion_matrix.asp
# Sharma, A. (2026, January 23). Understanding the Confusion Matrix in Machine Learning. Geeksforgeeks. https://www.geeksforgeeks.org/machine-learning/confusion-matrix-machine-learning/
# Galarnyk, M. (2025, August 4). Train Test Split: What It Means and How to Use It. Built In. https://builtin.com/data-science/train-test-split
```

```
tdata = pd.read_csv('train.csv')

# This is to calculate the diff_sym_hos from symptom onset and hospital visit dates
tdata['sym_on'] = pd.to_datetime(tdata['sym_on'], errors='coerce')
tdata['hosp_vis'] = pd.to_datetime(tdata['hosp_vis'], errors='coerce')
tdata['diff_sym_hos'] = (tdata['hosp_vis'] - tdata['sym_on']).dt.days
tdata['diff_sym_hos'] = tdata['diff_sym_hos'].apply(lambda x: x if x >= 0 else np.nan)
tdata['diff_sym_hos'] = tdata['diff_sym_hos'].fillna(tdata['diff_sym_hos'].median())

# This is for the numerical & Binary Processing
tdata['age'] = pd.to_numeric(tdata['age'], errors='coerce')
tdata['age'] = tdata['age'].fillna(tdata['age'].median())

binary_cols = ['vis_wuhan', 'from_wuhan', 'death', 'recov']
for col in binary_cols:
    if col in tdata.columns:
        tdata[col] = pd.to_numeric(tdata[col], errors='coerce').fillna(0).astype(int)

# This is for the label encoding and text normalization of the categorical columns
cat_cols = ['location', 'country', 'gender', 'symptom1', 'symptom2', 'symptom3', 'symptom4', 'symptom5', 'symptom6']
le = LabelEncoder()
for col in cat_cols:
    if col in tdata.columns:
        tdata[col] = tdata[col].astype(str).str.strip().str.lower()
        tdata[col] = tdata[col].replace({'nan': 'na', '': 'na', 'none': 'na', 'null': 'na'}).fillna('na')
        tdata[col] = le.fit_transform(tdata[col])

# evaluation and training of the model using the Boosted Random Forest
X = tdata[['location', 'country', 'gender', 'age', 'vis_wuhan', 'from_wuhan', 'symptom1', 'symptom2', 'symptom3', 'symptom4', 'symptom5', 'symptom6', 'diff_sym_hos']]
Y = tdata['death']

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=10)

# This initializes and fits the Boosted Random Forest Classifier
base_estimator = RandomForestClassifier(max_depth=2, random_state=42)
classifier = AdaBoostClassifier(estimator=base_estimator, n_estimators=50, random_state=42)
classifier.fit(X_train, np.array(Y_train).reshape(Y_train.shape[0], 1).ravel())

# This is the model evaluation for the Boosted Random Forest Classifier
pred = np.array(classifier.predict(X_test))

recall_brf = rs(Y_test, pred, zero_division=0)
precision_brf = ps(Y_test, pred, zero_division=0)
f1_brf = fs(Y_test, pred, zero_division=0)
ma_brf = classifier.score(X_test, Y_test)

print(" ")
print("="*70)
print('Evaluation metrics for test dataset (Boosted Random Forest)')
print("="*70)
print('Recall Score: ', recall_brf)
print('Precision Score: ', precision_brf)
print('F1 Score: ', f1_brf)
print('Accuracy: ', ma_brf)

a = pd.DataFrame(Y_test)
a['pred'] = classifier.predict(X_test)
print('\n\tTable 3\n')
print(a.head())

# this calculates and extracts Confusion Matrix values
cm_brf = confusion_matrix(Y_test, pred)
TN_brf, FP_brf, FN_brf, TP_brf = cm_brf.ravel()

print(" ")
print("="*50)
print(' Confusion Matrix (Boosted Random Forest) ')
print("="*50)
print(f"True Negatives (TN):          {TN_brf}")
print(f"False Positives (FP):         {FP_brf}")
print(f"False Negatives (FN):         {FN_brf}")
print(f"True Positives (TP):          {TP_brf}")

# This is for the plot
plt.bar(['Accuracy','F1 Score','Recall Score','Precision Score'],[ma_brf,f1_brf,recall_brf,precision_brf],color=['red','green','purple','orange'])
```

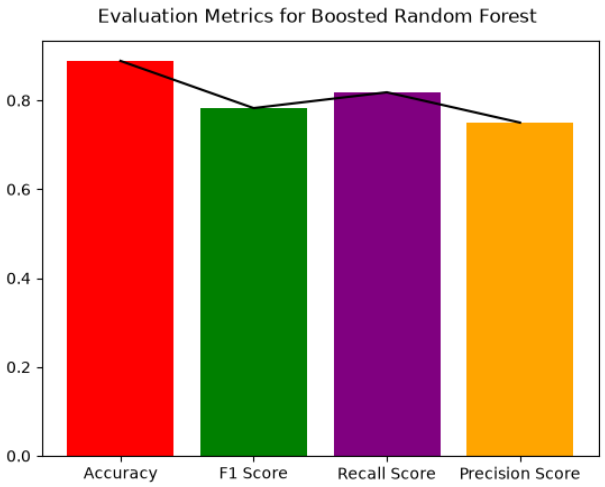
```
plt.plot([ma_brf,f1_brf,recall_brf,precision_brf],color='black')
plt.title('Evaluation Metrics for Boosted Random Forest ',fontsize=12, pad=12)
plt.show()
```

```
======================================================================
Evaluation metrics for test dataset (Boosted Random Forest)
======================================================================
Recall Score:  0.8181818181818182
Precision Score:  0.75
F1 Score:  0.782608695652174
Accuracy:  0.8888888888888888
```

Table 3

	death	pred
184	0	0
170	1	1
142	0	0
182	0	0
49	1	1

```
================================================
 Confusion Matrix (Boosted Random Forest)
================================================
True Negatives (TN):          31
False Positives (FP):         3
False Negatives (FN):         2
True Positives (TP):          9
```



4.4 Gaussian Naive Bayes

In [142…

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import recall_score as rs, precision_score as ps, f1_score as fs
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix

# Same references were used throughout the evaluation metrics section
# These are the references for the boosted random forest and the evaluation metrics
# Aman, S. (2026, March 30). Evaluation Metrics in Machine Learning . Geeksforgeeks. https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/
# Peshkar, A. (2020). Covid-19-Patient-Health-Analytics. Github. https://github.com/Peshkar/Covid-19-Patient-Health-Analytics
# Scikit. (2026). 3.4. Metrics and scoring: quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html
# Scikit. (2026). DecisionTreeClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html
# Scikit. (2026). train_test_split. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html
# shahjamanbadhon.(2025). Machine-learning-practice. Github. https://github.com/shahjamanbadhon/machine-learning-practice/blob/main/Confusion_Matrix.ipynb
```

```
# W3Schools. (2026). Python Machine Learning - Confusion Matrix. https://www.w3schools.com/python/python_ml_confusion_matrix.asp
# Sharma, A. (2026, January 23). Understanding the Confusion Matrix in Machine Learning. Geeksforgeeks. https://www.geeksforgeeks.org/machine-learning/confusion-matrix-machine-learning/
# Galarnyk, M. (2025, August 4). Train Test Split: What It Means and How to Use It. Built In. https://builtin.com/data-science/train-test-split

tdata = pd.read_csv('train.csv')

# This is to calculate the diff_sym_hos from symptom onset and hospital visit dates
tdata['sym_on'] = pd.to_datetime(tdata['sym_on'], errors='coerce')
tdata['hosp_vis'] = pd.to_datetime(tdata['hosp_vis'], errors='coerce')
tdata['diff_sym_hos'] = (tdata['hosp_vis'] - tdata['sym_on']).dt.days
tdata['diff_sym_hos'] = tdata['diff_sym_hos'].apply(lambda x: x if x >= 0 else np.nan)
tdata['diff_sym_hos'] = tdata['diff_sym_hos'].fillna(tdata['diff_sym_hos'].median())

# The Numerical & Binary Processing
tdata['age'] = pd.to_numeric(tdata['age'], errors='coerce')
tdata['age'] = tdata['age'].fillna(tdata['age'].median())

binary_cols = ['vis_wuhan', 'from_wuhan', 'death', 'recov']
for col in binary_cols:
    if col in tdata.columns:
        tdata[col] = pd.to_numeric(tdata[col], errors='coerce').fillna(0).astype(int)

# The Categorical text normalization and tabel encoding
cat_cols = ['location', 'country', 'gender', 'symptom1', 'symptom2', 'symptom3', 'symptom4', 'symptom5', 'symptom6']
le = LabelEncoder()
for col in cat_cols:
    if col in tdata.columns:
        tdata[col] = tdata[col].astype(str).str.strip().str.lower()
        tdata[col] = tdata[col].replace({'nan': 'na', '': 'na', 'none': 'na', 'null': 'na'}).fillna('na')
        tdata[col] = le.fit_transform(tdata[col])

# training adn evaluation of the model using the Gaussian Naive Bayes Classifier
X = tdata[['location', 'country', 'gender', 'age', 'vis_wuhan', 'from_wuhan', 'symptom1', 'symptom2', 'symptom3', 'symptom4', 'symptom5', 'symptom6', 'diff_sym_hos']]
Y = tdata['death']

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=10)

# This initializes and fits the Gaussian Naive Bayes Classifier
classifier = GaussianNB()
classifier.fit(X_train, np.array(Y_train).reshape(Y_train.shape[0], 1).ravel())

# The evaluation of the model
pred = np.array(classifier.predict(X_test))

recall_gnb = rs(Y_test, pred, zero_division=0)
precision_gnb = ps(Y_test, pred, zero_division=0)
f1_gnb = fs(Y_test, pred, zero_division=0)
ma_gnb = classifier.score(X_test, Y_test)


print(" ")
print("="*70)
print(' Evaluation metrics for test dataset (Gaussian Naive Bayes)')
print("="*70)
print('Recall Score: ', recall_gnb)
print('Precision Score: ', precision_gnb)
print('F1 Score: ', f1_gnb)
print('Accuracy: ', ma_gnb)

a = pd.DataFrame(Y_test)
a['pred'] = classifier.predict(X_test)
print('\n\tTable 3\n')
print(a.head())

# This calculates confusion matrix and extracts the values
cm_gnb = confusion_matrix(Y_test, pred)
TN_gnb, FP_gnb, FN_gnb, TP_gnb = cm_gnb.ravel()

print(" ")
print("="*50)
print(' Confusion Matrix (Gaussian Naive Bayes) ')
print("="*50)
print(f"True Negatives (TN):          {TN_gnb}")
print(f"False Positives (FP):         {FP_gnb}")
print(f"False Negatives (FN):         {FN_gnb}")
```

```
print(f"True Positives (TP):          {TP_gnb}")

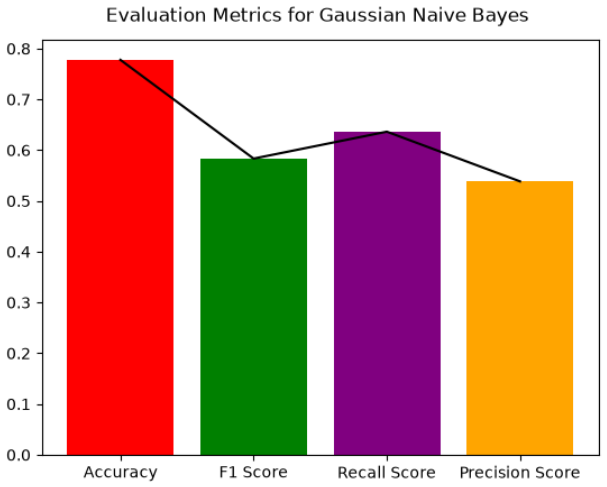
# This is for the plot
plt.bar(['Accuracy','F1 Score','Recall Score','Precision Score'],[ma_gnb,f1_gnb,recall_gnb,precision_gnb],color=['red','green','purple','orange'])
plt.plot([ma_gnb,f1_gnb,recall_gnb,precision_gnb],color='black')
plt.title('Evaluation Metrics for Gaussian Naive Bayes ',fontsize=12, pad=12)
plt.show()
```

```
====================================================================
 Evaluation metrics for test dataset (Gaussian Naive Bayes)
====================================================================
Recall Score:  0.6363636363636364
Precision Score:  0.5384615384615384
F1 Score:  0.5833333333333334
Accuracy:  0.7777777777777778

        Table 3

     death  pred
184      0     0
170      1     0
142      0     0
182      0     0
49       1     1

================================================
 Confusion Matrix (Gaussian Naive Bayes)
================================================
True Negatives (TN):          28
False Positives (FP):         6
False Negatives (FN):         4
True Positives (TP):          7
```



4.5 Support Vector Machine

In [143]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import recall_score as rs, precision_score as ps, f1_score as fs
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix

# Same references were used throughout the evaluation metrics section
# These are the references for the boosted random forest and the evaluation metrics
# Aman, S. (2026, March 30). Evaluation Metrics in Machine Learning . Geeksforgeeks. https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/
# Peshkar, A. (2020). Covid-19-Patient-Health-Analytics. Github. https://github.com/Peshkar/Covid-19-Patient-Health-Analytics
```

```
# Scikit. (2026). 3.4. Metrics and scoring: quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html
# Scikit. (2026). DecisionTreeClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html
# Scikit. (2026). train_test_split. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html
# shahjamanbadhon.(2025). Machine-learning-practice. Github. https://github.com/shahjamanbadhon/machine-learning-practice/blob/main/Confusion_Matrix.ipynb
# W3Schools. (2026). Python Machine Learning - Confusion Matrix. https://www.w3schools.com/python/python_ml_confusion_matrix.asp
# Sharma, A. (2026, January 23). Understanding the Confusion Matrix in Machine Learning. Geeksforgeeks. https://www.geeksforgeeks.org/machine-learning/confusion-matrix-machine-learning/
# Galarnyk, M. (2025, August 4). Train Test Split: What It Means and How to Use It. Built In. https://builtin.com/data-science/train-test-split


tdata = pd.read_csv('train.csv')
# This is to calculate the diff_sym_hos from symptom onset and hospital visit dates
tdata['sym_on'] = pd.to_datetime(tdata['sym_on'], errors='coerce')
tdata['hosp_vis'] = pd.to_datetime(tdata['hosp_vis'], errors='coerce')
tdata['diff_sym_hos'] = (tdata['hosp_vis'] - tdata['sym_on']).dt.days
tdata['diff_sym_hos'] = tdata['diff_sym_hos'].apply(lambda x: x if x >= 0 else np.nan)
tdata['diff_sym_hos'] = tdata['diff_sym_hos'].fillna(tdata['diff_sym_hos'].median())

# This is for the numerical & Binary Processing
tdata['age'] = pd.to_numeric(tdata['age'], errors='coerce')
tdata['age'] = tdata['age'].fillna(tdata['age'].median())

binary_cols = ['vis_wuhan', 'from_wuhan', 'death', 'recov']
for col in binary_cols:
    if col in tdata.columns:
        tdata[col] = pd.to_numeric(tdata[col], errors='coerce').fillna(0).astype(int)

# This is for categorical text normalization & label encoding
cat_cols = ['location', 'country', 'gender', 'symptom1', 'symptom2', 'symptom3', 'symptom4', 'symptom5', 'symptom6']
le = LabelEncoder()
for col in cat_cols:
    if col in tdata.columns:
        tdata[col] = tdata[col].astype(str).str.strip().str.lower()
        tdata[col] = tdata[col].replace({'nan': 'na', '': 'na', 'none': 'na', 'null': 'na'}).fillna('na')
        tdata[col] = le.fit_transform(tdata[col])

# Model training adn evaluation#
X = tdata[['location', 'country', 'gender', 'age', 'vis_wuhan', 'from_wuhan', 'symptom1', 'symptom2', 'symptom3', 'symptom4', 'symptom5', 'symptom6', 'diff_sym_hos']]
Y = tdata['death']

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=10)

# This initializes and fits the Support Vector Classifier
classifier = SVC(random_state=42)
classifier.fit(X_train, np.array(Y_train).reshape(Y_train.shape[0], 1).ravel())

# this is for the Evaluation of the model
pred = np.array(classifier.predict(X_test))

recall_svm = rs(Y_test, pred, zero_division=0)
precision_svm = ps(Y_test, pred, zero_division=0)
f1_svm = fs(Y_test, pred, zero_division=0)
ma_svm = classifier.score(X_test, Y_test)

print(" ")
print("="*70)
print(' Evaluation metrics for test dataset (Support Vector Machine) ')
print("="*70)
print('Recall Score: ', recall_svm)
print('Precision Score: ', precision_svm)
print('F1 Score: ', f1_svm)
print('Accuracy: ', ma_svm)

a = pd.DataFrame(Y_test)
a['pred'] = classifier.predict(X_test)
print('\n\tTable 3\n')
print(a.head())

# This Calculates and extracts the Confusion Matrix (TP, TN, FP, FN)
cm_svm = confusion_matrix(Y_test, pred)
TN_svm, FP_svm, FN_svm, TP_svm = cm_svm.ravel()

print(" ")
print("="*50)
print(' Confusion Matrix (Support Vector Machine) ')
print("="*50)
```

```
print(f"True Negatives (TN):          {TN_svm}")
print(f"False Positives (FP):         {FP_svm}")
print(f"False Negatives (FN):         {FN_svm}")
print(f"True Positives (TP):          {TP_svm}")

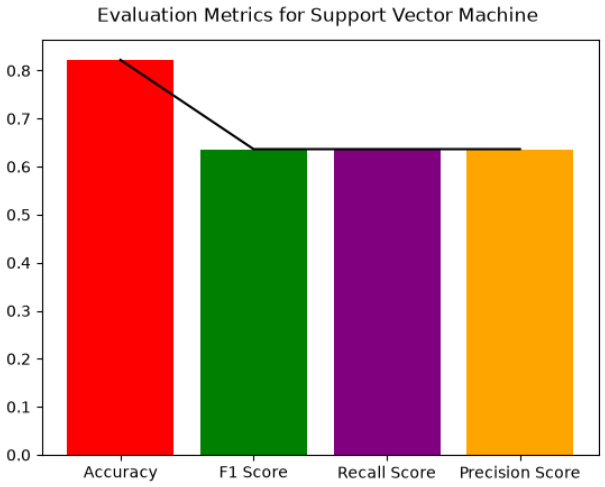
# This is for the plot
plt.bar(['Accuracy','F1 Score','Recall Score','Precision Score'],[ma_svm,f1_svm,recall_svm,precision_svm],color=['red','green','purple','orange'])
plt.plot([ma_svm,f1_svm,recall_svm,precision_svm],color='black')
plt.title('Evaluation Metrics for Support Vector Machine ',fontsize=12, pad=12)
plt.show()
```

```
================================================================
 Evaluation metrics for test dataset (Support Vector Machine)
================================================================
Recall Score:  0.6363636363636364
Precision Score:  0.6363636363636364
F1 Score:  0.6363636363636364
Accuracy:  0.8222222222222222

        Table 3

     death  pred
184      0     0
170      1     0
142      0     0
182      0     0
49       1     1

================================================
 Confusion Matrix (Support Vector Machine)
================================================
True Negatives (TN):          30
False Positives (FP):         4
False Negatives (FN):         4
True Positives (TP):          7
```



4.6 Decision Trees (Initial, 1, 10, 25 & 100)

In [144…

```
import warnings
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.model_selection import GridSearchCV, StratifiedKFold, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

```
warnings.filterwarnings("ignore", category=UserWarning)
warnings.filterwarnings("ignore", category=RuntimeWarning)

# These are the references
# Peshkar, A. (2020). Covid-19-Patient-Health-Analytics. Github. https://github.com/Peshkar/Covid-19-Patient-Health-Analytics
# Galarnyk, M. (2020). Decision Trees Visualization. Github. https://github.com/mGalarnyk/Python_Tutorials/blob/master/Sklearn/CART/Visualization/DecisionTreesVisualization.ipynb
# Scikit. (n.d.). Understanding the decision tree structure. https://scikit-learn.org/stable/auto_examples/tree/plot_unveil_tree_structure.html
# Scikit. (n.d.). Preprocessing data. https://scikit-learn.org/stable/modules/preprocessing.html

# The is to load and preprocess the data, including calculating the difference between symptom onset and hospital visit dates, handling missing values, and encoding categorical variables.
dataset = pd.read_csv("train.csv")

# This is the code for cleaning, formating and categorizing of the data
dataset["sym_on"] = pd.to_datetime(dataset["sym_on"], errors="coerce")
dataset["hosp_vis"] = pd.to_datetime(dataset["hosp_vis"], errors="coerce")
dataset["diff_sym_hos"] = (dataset["hosp_vis"] - dataset["sym_on"]).dt.days
dataset["diff_sym_hos"] = dataset["diff_sym_hos"].apply(
    lambda x: x if x >= 0 else np.nan
)
dataset["diff_sym_hos"] = dataset["diff_sym_hos"].fillna(
    dataset["diff_sym_hos"].median()
)
dataset["age"] = pd.to_numeric(dataset["age"], errors="coerce")
dataset["age"] = dataset["age"].fillna(dataset["age"].median())

binary_cols = ["vis_wuhan", "from_wuhan", "death", "recov"]
for col in binary_cols:
  if col in dataset.columns:
    dataset[col] = (
        pd.to_numeric(dataset[col], errors="coerce").fillna(0).astype(int)
    )

cat_cols = [
    "location",
    "country",
    "gender",
    "symptom1",
    "symptom2",
    "symptom3",
    "symptom4",
    "symptom5",
    "symptom6",
]
le = LabelEncoder()
for col in cat_cols:
  if col in dataset.columns:
    dataset[col] = dataset[col].astype(str).str.strip().str.lower()
    dataset[col] = (
        dataset[col]
        .replace({"nan": "na", "": "na", "none": "na", "null": "na"})
        .fillna("na")
    )
    dataset[col] = le.fit_transform(dataset[col])

X = dataset[
    [
        "location",
        "country",
        "gender",
        "age",
        "vis_wuhan",
        "from_wuhan",
        "symptom1",
        "symptom2",
        "symptom3",
        "symptom4",
        "symptom5",
        "symptom6",
        "diff_sym_hos",
    ]
]
Y = dataset["death"]

# This is where the coloumn names are changed (Country = x1)
```

```
feature_labels = [f"X{i}" for i in range(X.shape[1])]

X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=10, stratify=Y
)

# This is the code for Hyperparameter Tuning using GridSearchCV & Pipeline
selector = SelectKBest(score_func=f_classif)
clf = RandomForestClassifier(random_state=42)

pipeline = Pipeline(
    [("feature_selection", selector), ("classification", clf)]
)

param_grid = {
    "feature_selection__k": [5, 8, 10, "all"],
    "classification__n_estimators": [50, 100],
    "classification__criterion": ["gini", "entropy"],
    "classification__max_depth": [3, 5, 10, None],
    "classification__min_samples_split": [2, 5, 10],
}

cv_strategy = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=cv_strategy,
    scoring="f1",
    n_jobs=-1,
)

grid_search.fit(X_train, Y_train)

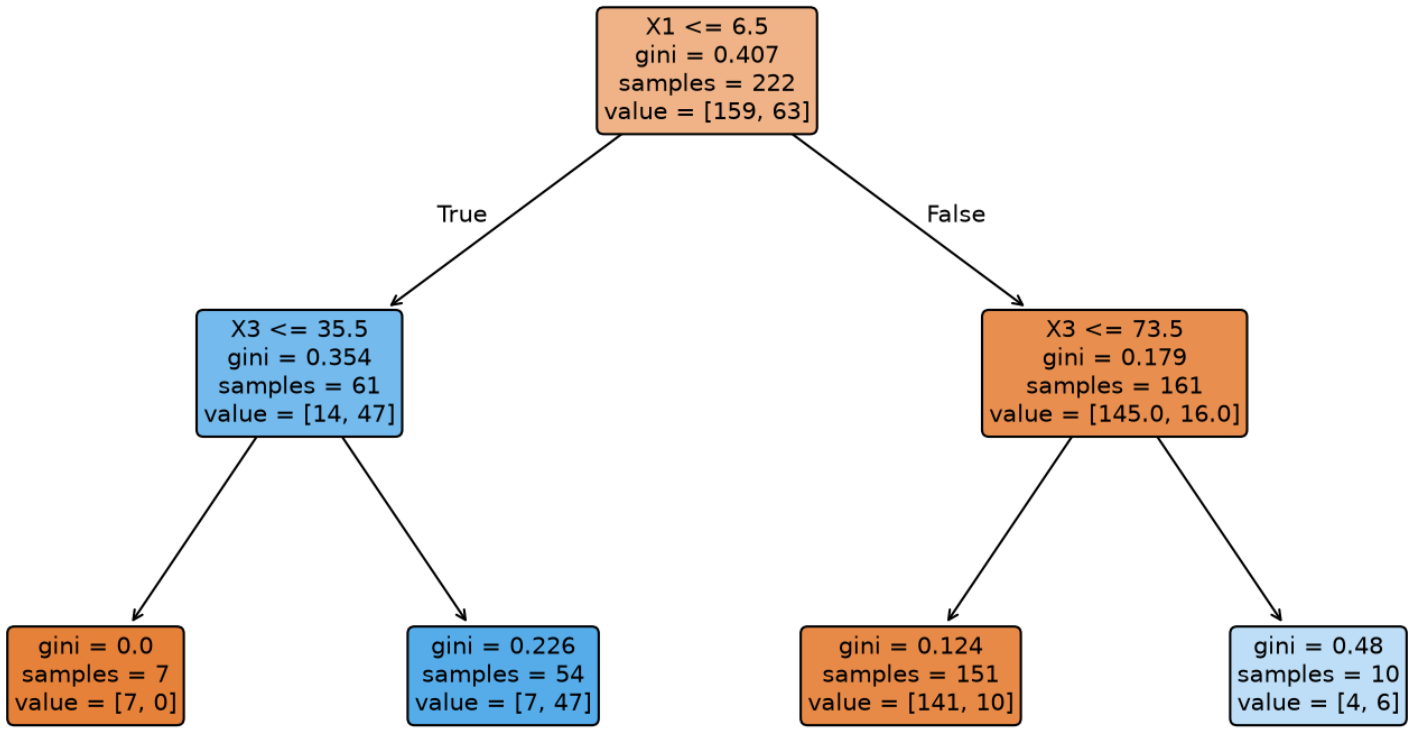
# This is the iinitial decision tree
# This is for the decision tree visualization (Initial Decision Tree with max_depth=2)
# Agarwal,Y. (2025, July 23). How to Visualize a Decision Tree from a Random Forest. https://www.geeksforgeeks.org/machine-learning/ways-to-visualize-individual-decision-trees-in-a-random-forest/
dt_clean = DecisionTreeClassifier(max_depth=2, random_state=42)
dt_clean.fit(X, Y)

plt.figure(figsize=(10, 6), dpi=150)
plot_tree(
    dt_clean,
    feature_names=feature_labels,
    filled=True,
    rounded=True,
    fontsize=10,
)
plt.title("Decision Tree (Initial) ", fontsize=13, pad=12)
plt.tight_layout()
plt.show()

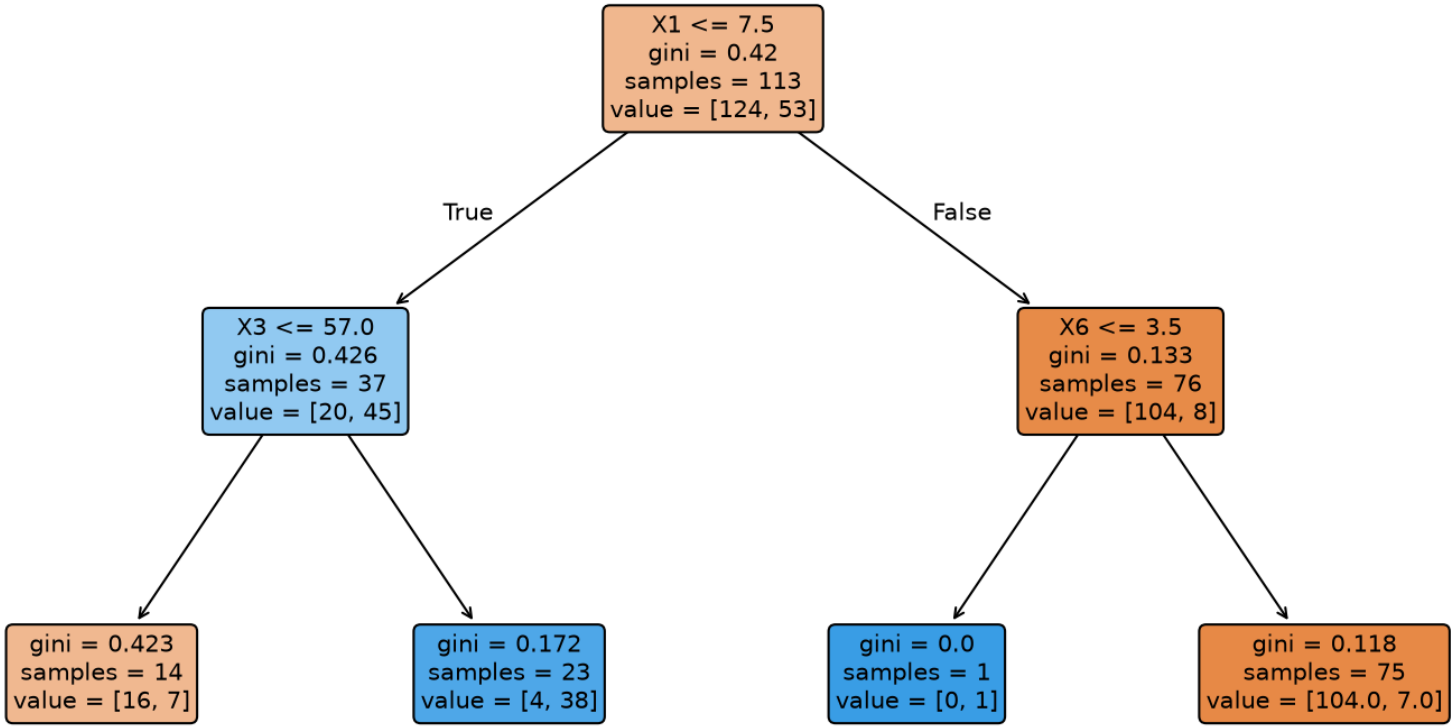
# Navlani, A. (2026, January 16). Decision Tree Classification in Python Tutorial. https://www.datacamp.com/tutorial/decision-tree-classification-python
# Train one Random Forest with 101 trees, then plot different individual trees from inside it
rf = RandomForestClassifier(n_estimators=101, max_depth=2, random_state=10)
rf.fit(X_train, Y_train)

tree_indices = [1, 10, 25, 100]

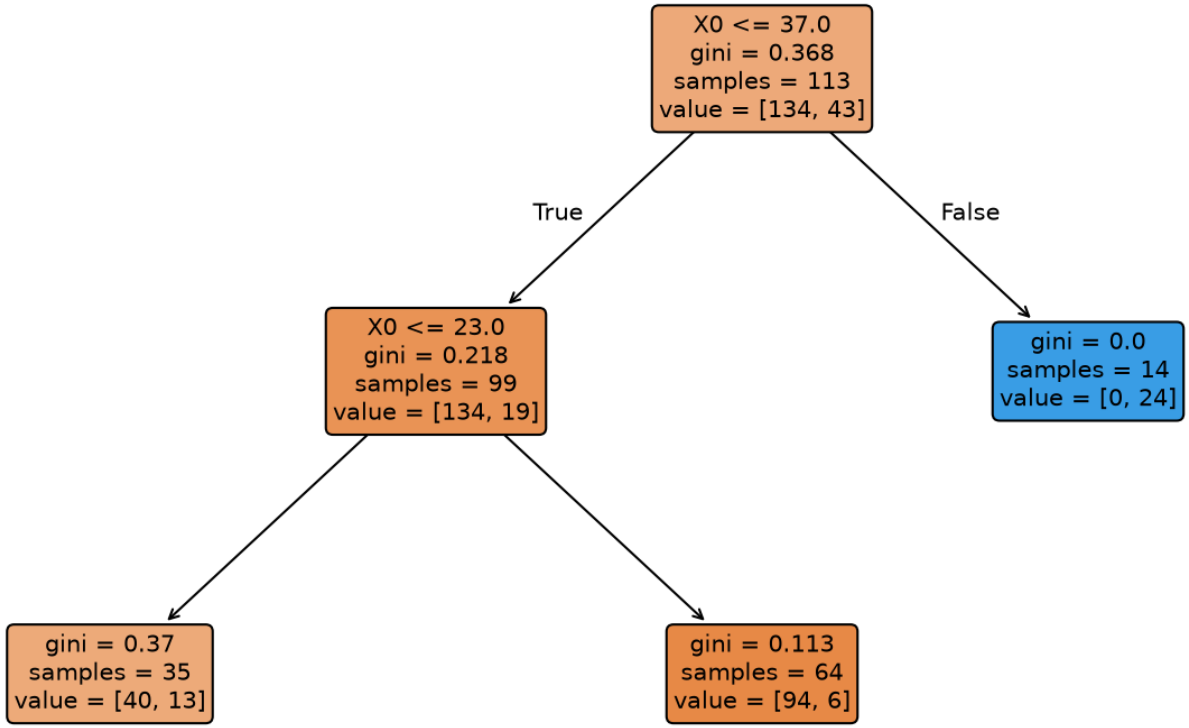
for idx in tree_indices:
  plt.figure(figsize=(10, 6), dpi=150)
  plot_tree(
      rf.estimators_[idx],
      feature_names=feature_labels,
      filled=True,
      rounded=True,
      fontsize=10,
  )
  plt.title(
      f"Decision Tree (n_estimator {idx})",
      fontsize=13,
      pad=12,
  )
  plt.tight_layout()
  plt.show()
```


Decision Tree (Initial)
X1 <= 6.5
gini = 0.407
samples = 222
value = [159, 63]
True
False
X3 <= 35.5
gini = 0.354
samples = 61
value = [14, 47]
X3 <= 73.5
gini = 0.179
samples = 161
value = [145.0, 16.0]
gini = 0.0
samples = 7
value = [7, 0]
gini = 0.226
samples = 54
value = [7, 47]
gini = 0.124
samples = 151
value = [141, 10]
gini = 0.48
samples = 10
value = [4, 6]

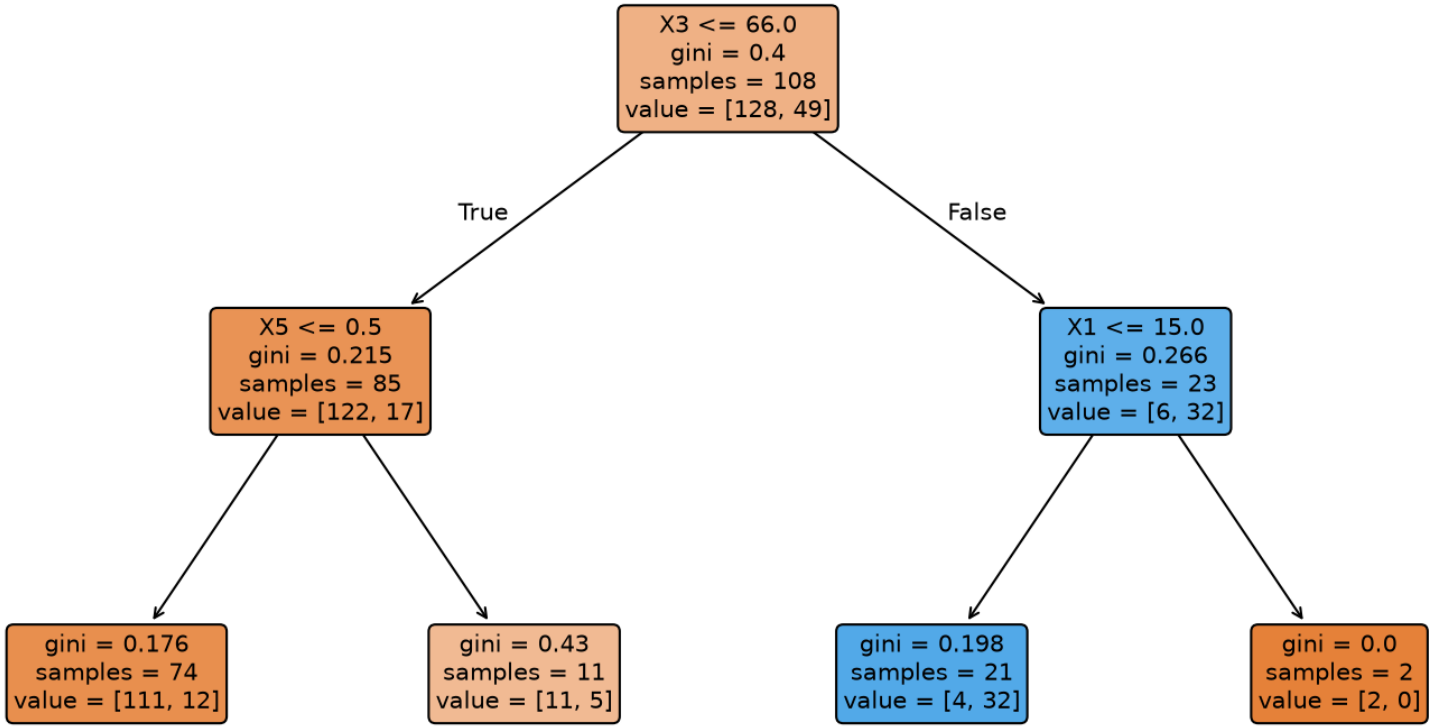
Decision Tree (n_estimator 1)

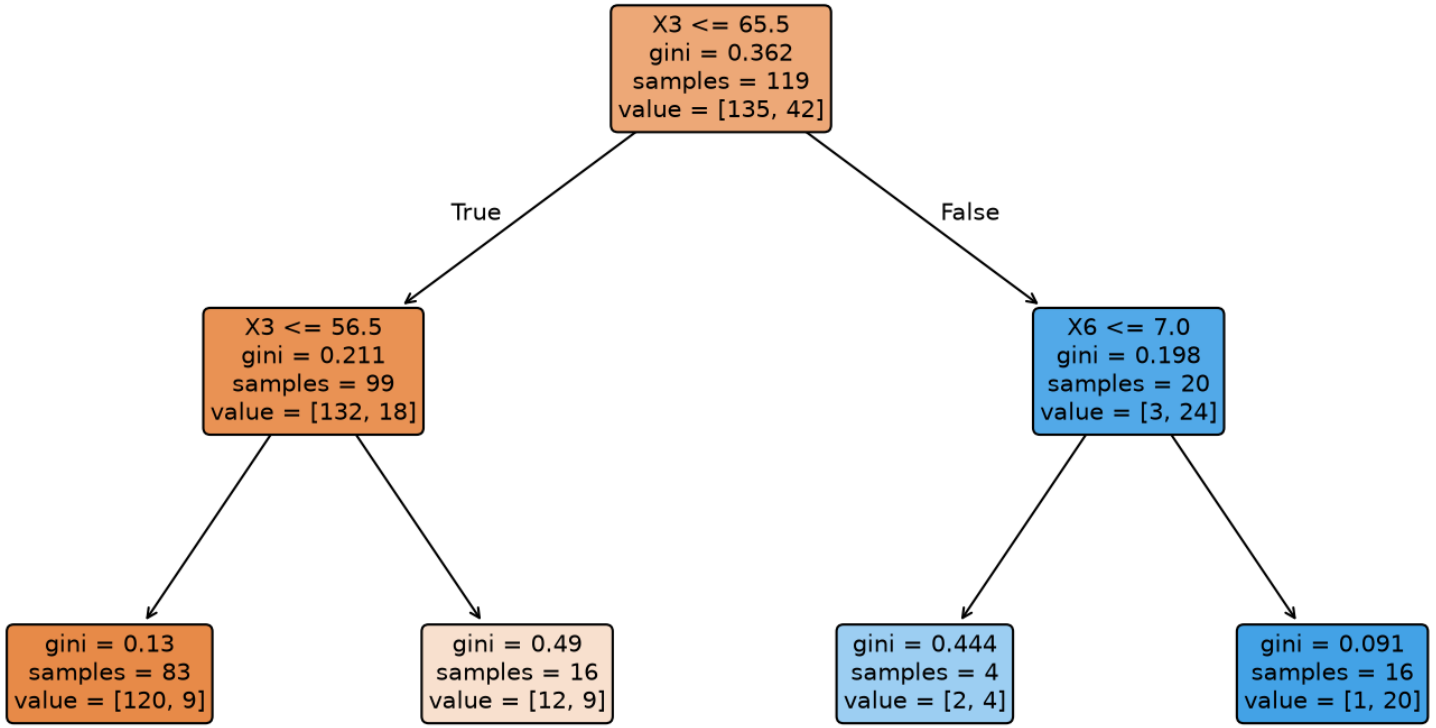


Decision Tree (n_estimator 10)



Decision Tree (n_estimator 25)





Evaluation Metrics:

- **Model comparisons:** 5 machine learning models were tested which were, `Descision tree`, `Boosted random forest`, `Gaussian naive bayes`, `Support vector machine` and `Logistic regression`, their accuracy, precision, recall, and F1 scores were looked at to find the best model.
- **The TOP Model:** The evaluation assisted in identifying that the Boosted Random Forest was the best because of the accuracy (89%) and F1(78%) scores
- **Issue adn solution:** The data missed alot of values which resulted in the neeed to use the `F1` scores as well as the initial `accuracy` score
- **Boosted random forest vs the descision tree:** `Descision tree` has low recall but high precision while `boosted random forest` had both high values which resulted in it being chosen.

5. Results:

- The correlation matrix displayed a strong correlation between these variables, `from_Wuhan` and `death`, `age` and `death` and `diff_sym_hos` and `death`, there was a high negative correlation between `symptom5` and `symptom6` but that is because 99% of their data is missing.
- The evaluation metrics identified that `Booster random forest` was the best model from its confusion matrix scores and evaluation metrics values but mostly from its `F1 score` due to the data being imbalanced.

- The decision trees gini score remained less that 0.5 which showed that there was imbalance in the training data used
- The optimal parameter that yielded the top performance for the model were a `criterion` of 'gini', `max depth` of 10, `min sample leaf` of 1, `min sample split` of 5 and the `n_estimators` of 200

6. Hyperparameter optimisation:

In [145]:
```
from sklearn.model_selection import GridSearchCV, StratifiedKFold, train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import pandas as pd
import numpy as np
# Singh, T. (2026, June 26). Hyperparameter Tuning. Geeksforgeeks. https://www.geeksforgeeks.org/machine-learning/hyperparameter-tuning/
# lakshaymbnwg. (2025, July 23). Performing Feature Selection with gridsearchcv in Sklearn. https://www.geeksforgeeks.org/machine-learning/performing-feature-selection-with-gridsearchcv-in-sklearn/
# Scikit. (n.d.). GridSearchCV. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
# Scikit. (n.d.). Random Forest Classifier. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html
#https://scikit-learn.org/stable/modules/model_evaluation.html
# Scikit. (n.d.). 3.2. Tuning the hyper-parameters of an estimator. https://scikit-learn.org/stable/modules/grid_search.html
# Scikit. (n.d.). 3.4. Metrics and scoring: quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html


# This is for defining the features and target variable, splitting the dataset into training and testing sets, and setting up the Random Forest Classifier with hyperparameter tuning using GridSearchCV.
X = tdata[['location', 'country', 'gender', 'age', 'vis_wuhan', 'from_wuhan', 'symptom1', 'symptom2', 'symptom3', 'symptom4', 'symptom5', 'symptom6', 'diff_sym_hos']]
Y = tdata['death']
X_train, X_test, Y_train, Y_test = train_test_split( X, Y, test_size=0.2, random_state=42, stratify=Y )
# This is for the base classifier
rf = RandomForestClassifier(random_state=42)

# this is for the parameters
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [2, 5, 10, None],
    'min_samples_leaf': [1, 2, 4],
    'min_samples_split': [2, 5],
    'criterion': ['gini', 'entropy']
}

# Thsi sets up the GridSearchCV
grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    scoring='accuracy',
    n_jobs=-1
)

# Fit the grid search to your training data
grid_search.fit(X_train, Y_train)

# This extract and display the Optimal hyperparameters returned by grid search
hyperparameter_data = {
    "Parameters": list(grid_search.best_params_.keys()),
    "Value": list(grid_search.best_params_.values())
}
hyperparameter_df = pd.DataFrame(hyperparameter_data)
print("="*50)
print("Optimal hyperparameters returned by grid search.")
print("="*50)
print(hyperparameter_df.to_string(index=False))
```

```
==================================================
Optimal hyperparameters returned by grid search.
==================================================
       Parameters Value
        criterion  gini
        max_depth    10
 min_samples_leaf     1
min_samples_split     5
     n_estimators   200
```

Hyperparameter optimisation:

- **Model testing:** Different parameters were tested to find the best configuration of the models that boosted the scores and reduced/prevented overfitting.

7. Comparison of models performance:

In [146...

```python
import pandas as pd
import matplotlib.pyplot as plt


# These are the references
# Peshkar, A. (2020). Covid-19-Patient-Health-Analytics. Github. https://github.com/Peshkar/Covid-19-Patient-Health-Analytics
# Venmani A D. (n.d.). Matplotlib Line Plot – How to create a line plot to visualize the trend?. Machine Learning Plus. https://machinelearningplus.com/plots/matplotlib-line-plot/
# Bobbit, Z. (2022, April 6). How to Plot Multiple Lines in Matplotlib. Statology. https://www.statology.org/plot-multiple-lines-matplotlib/
# Dhingra, F. ( 2025, July 23). How to plot multiple data columns in a DataFrame?. Geeksforgeeks. https://www.geeksforgeeks.org/how-to-plot-multiple-data-columns-in-a-dataframe/
# Pandas. 2026. pandas.DataFrame.plot. https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.plot.html

# Manually input the exact evaluation metrics fro each model for the plot
evaluation_data = [
    {
        "Model": "Logistic Regression",
        "Accuracy": 0.8444444444444444,
        "Precision": 0.6666666666666666,
        "Recall": 0.7272727272727273,
        "F1 Score": 0.6956521739130435
    },
    {
        "Model": "Decision Tree",
        "Accuracy": 0.8666666666666667,
        "Precision": 0.8571428571428571,
        "Recall": 0.5454545454545454,
        "F1 Score": 0.6666666666666666
    },
    {
        "Model": "Boosted Random Forest",
        "Accuracy": 0.8888888888888888,
        "Precision": 0.75,
        "Recall": 0.8181818181818182,
        "F1 Score": 0.782608695652174
    },
    {
        "Model": "SVM",
        "Accuracy": 0.8222222222222222,
        "Precision": 0.6363636363636364,
        "Recall": 0.6363636363636364,
        "F1 Score": 0.6363636363636364
    },
    {
        "Model": "Gaussian NB",
        "Accuracy": 0.7777777777777778,
        "Precision": 0.5384615384615384,
        "Recall": 0.6363636363636364,
        "F1 Score": 0.5833333333333334
    }
]

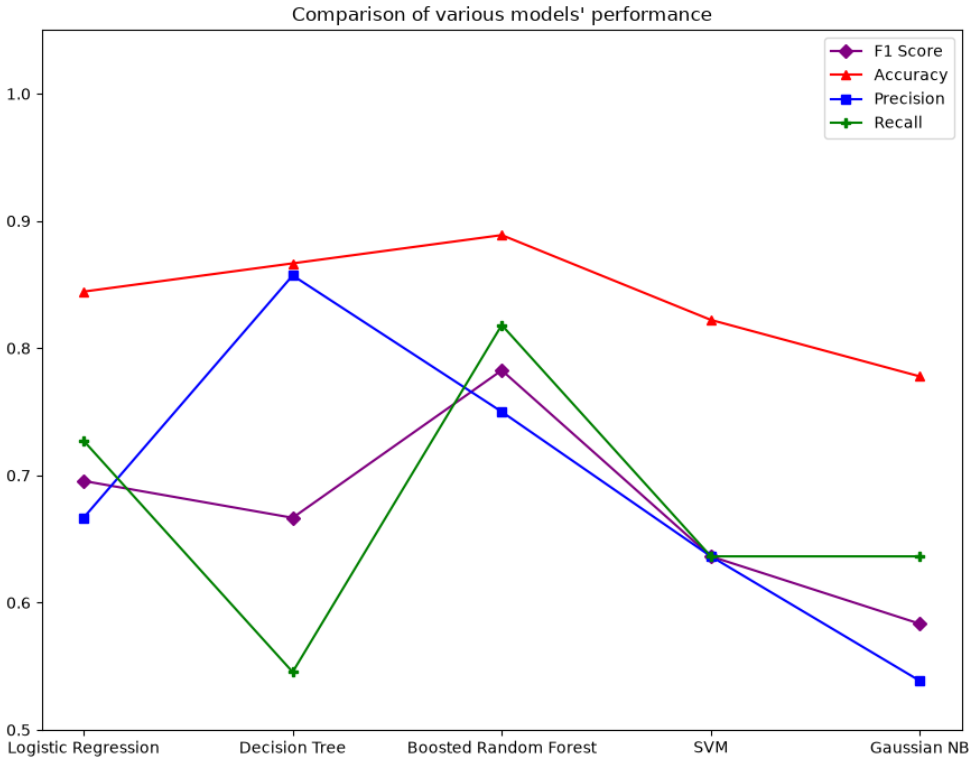
comparison_df = pd.DataFrame(evaluation_data)
display(comparison_df)

# this Plots line chart matching the target layout and styling
plt.figure(figsize=(9, 7))

plt.plot(comparison_df["Model"], comparison_df["F1 Score"], marker='D', color='purple', label='F1 Score')
plt.plot(comparison_df["Model"], comparison_df["Accuracy"], marker='^', color='red', label='Accuracy')
plt.plot(comparison_df["Model"], comparison_df["Precision"], marker='s', color='blue', label='Precision')
plt.plot(comparison_df["Model"], comparison_df["Recall"], marker='P', color='green', label='Recall')

plt.title("Comparison of various models' performance")
plt.ylim(0.5, 1.05)
plt.legend(loc='upper right')
plt.tight_layout()
plt.show()
```

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.844444	0.666667	0.727273	0.695652
1	Decision Tree	0.866667	0.857143	0.545455	0.666667
2	Boosted Random Forest	0.888889	0.750000	0.818182	0.782609
3	SVM	0.822222	0.636364	0.636364	0.636364
4	Gaussian NB	0.777778	0.538462	0.636364	0.583333



Comparison of models performance:

- **The best model:** The 5 models `F1 scores`, `accuracy`, `precision` and `recall` values were layered together to identify the best model, after observation and score comparison it was concluded that the `Boosted Random Forest` was the superior model out of the 5